

ENGINEERING A PAPER- FOLDING ROBOT

MTE 100 & MTE 121 Final Project

Group 6:

Connor Zhou (21124538), Jonathan Wang (21189809),

Jakob Patel (21215825), Noah Sun (21181420)

December 2, 2025

Abstract

This report presents the design and fabrication of a paper folding robot aimed at automating folds into perfect thirds. Creating perfect thirds on a sheet of paper by hand is a tedious, difficult, and time-consuming task. By automating this job, a robot can complete more sheets in less time than a human, while improving accuracy and consistency. The system was developed using VEX IQ components and custom 3D-printed mechanisms to meet project constraints limiting the design to four motors and one of each available sensor. A top-feeding indexer, four synchronized rollers, and two motor-driven rack-and-pinion hard stops enabled adjustable fold locations for creating perfect thirds on standard letter-sized paper. Additional sensors placed in critical locations provided jam detection, as well as recognizing an empty feeder tray, and full output tray. The final robot met all project criteria and constraints, creating folds within 0.5 cm of the desired distance due to the use of a PID, keeping the fold angle less than 5° from a sturdy and reliable frame, and completing folds without damaging the paper in under 10 seconds. These results showed that the robot was overall successful in executing its intended function.

Acknowledgements

We would like to acknowledge and extend our appreciation to the teaching team of MTE100 and MTE121.

We would like to thank Prof. Sanjeev Bedi, Prof. William Melek, and Prof. Abdullah Barakat for their guidance and assistance in the overall process, from project definition up to right before our demo. Additionally, we would also like to thank Prof. Ryan Consell for his help with the technical writing portion of the project. We would also like to extend our warmest appreciation to the teaching assistants and tutors of MTE100 for their continued support and assistance throughout the project. Notably, Daniel Xie for giving us feedback during the informal and formal presentation and finally, during Demo Day.

We would like to thank Prof. Ahmed Mostafa and Prof. Divya Lala for their support and guidance on the software portion of this project, answering our questions about the limitations of the software. Especially, Prof. Lala for all the help with the VEX programming and for providing feedback during Demo Day. Additionally, we would also like to extend our thanks to the teaching assistants of MTE121, notably Alice, who provided feedback and assistance during the software design meeting.

Table of Contents

1.0	Introduction.....	1
1.1.	Scope	1
1.2.	Constraints.....	1
1.3.	Criteria.....	2
2.0	Mechanical Design and Implementation.....	3
2.1.	Frame and Bracing	5
2.2.	Indexer.....	7
2.3.	Rollers.....	9
2.4.	Hardstops.....	11
2.5.	Output Tray	14
3.0	Software Design and Implementation.....	16
3.1.	Fold Location Input	16
3.1.1.	Button Presses.....	18
3.2.	Main Process	18
3.2.1.	Pause.....	19
3.2.2.	Display Error.....	20
3.3.	PID Controller	20
3.3.1.	Theory.....	21
3.3.2.	Tuning.....	22
3.3.3.	Implementation	23
3.4.	Motor	24
3.4.1.	Stall Detection.....	24
3.5.	Feeder	25
3.5.1.	Index	25
3.6.	Hardstop	26
3.6.1.	Calibration	26

3.6.2.	Control	27
3.7.	Rollers.....	27
3.8.	Task List	28
3.9.	Testing Procedure	29
3.10.	Problems Encountered.....	31
4.0	Verification	32
5.0	Project Plan	33
6.0	Conclusions.....	35
7.0	Recommendations.....	37
7.1.	Mechanical Design	37
7.2.	Software Design	37
References.....		39
Appendix A – C++ Code		40
main.cpp.....		40
rollers.cpp.....		47
rollers.h		48
hardstop.cpp.....		48
hardstop.h.....		49
pid_controller.cpp		50
pid_controller.h.....		51
motor.cpp		51
motor.h.....		51
feeder.cpp.....		52
feeder.h		53
util.h		53

List of Figures

FIGURE 1: HINGE DESIGN.....	3
FIGURE 2: ROBOT ARM	4
FIGURE 3: TWO-ARMED DESIGN.....	4
FIGURE 4: ROLLER DESIGN	4
FIGURE 5: OLD FRAME SIDE VIEW AND TOP VIEW	5
FIGURE 6: NEW FRAME SIDE VIEW AND TOP VIEW	6
FIGURE 7: SHAFT BRACING FOR HARD STOP PINIONS	6
FIGURE 8: INITIAL BOTTOM FEEDING INDEXER DESIGN USING RUBBER BANDS FOR GRIP	7
FIGURE 9: FINAL TOP FEEDING INDEXER DESIGN USING POLYURETHANE CONDOM FOR GRIP	9
FIGURE 10: ROLLER GEAR LAYOUT CAD	9
FIGURE 11: ROLLER CAD	10
FIGURE 12: HYBRID ROLLER.....	10
FIGURE 13: CRANKSHAFT [1]	12
FIGURE 14: GUIDED LEAD SCREW AXIS [2]	12
FIGURE 15: RACK AND PINION TRAY CAD	13
FIGURE 16: CURVED TRAY CAD.....	14
FIGURE 17: MAGNET HOLDERS.....	14
FIGURE 18: COMPLETED OUTPUT TRAY IN PLACE	15
FIGURE 19: CODE SNIPPET OF USING ARRAYS TO INCREMENT/DECREMENT FOLD LOCATIONS	17
FIGURE 20: FLOWCHART OF INPUTFOLDLOCATIONS()	18
FIGURE 21: FLOWCHART OF MAIN().....	19
FIGURE 22: FLOWCHART OF PAUSE()	19
FIGURE 23: FLOWCHART OF DISPLAYERROR().....	20
FIGURE 24: FLOWCHART OF PID.CALCULATE()	23
FIGURE 25: FLOWCHART OF FEEDER.INDEX()	25
FIGURE 26: FLOWCHART OF HARDSTOP.CALIBRATE()	27
FIGURE 27: INITIAL GANTT CHART	34

List of Tables

TABLE 1: OVERALL DESIGN DECISION MATRIX.....	3
TABLE 2: DECISION MATRIX FOR RUBBER BAND REPLACEMENT.....	8
TABLE 3: HARDSTOP MECHANISM DECISION MATRIX	11
TABLE 4: FUNCTION SUMMARY OF INPUTFOLDLOCATIONS.....	16
TABLE 5: FUNCTION SUMMARY OF PAUSE().....	19
TABLE 6: FUNCTION SUMMARY OF DISPLAYERROR()	20
TABLE 7: FUNCTION SUMMARY OF PID.CALCULATE()	23
TABLE 8: FUNCTION SUMMARY OF MOTOR.ISSTALLING()	25
TABLE 9: FUNCTION SUMMARY FOR FEEDER.INDEX().....	26
TABLE 10: FUNCTION SUMMARY OF HARDSTOP.CALIBRATE().....	27
TABLE 11: FUNCTION SUMMARY OF ROLLERS.ISOUTPUTTRAYFULL().....	28
TABLE 12: FUNCTION SUMMARY OF ROLLERS.ISCLEAR().....	28
TABLE 13: FINAL TASKS LIST USED DURING DEMO DAY	28
TABLE 14: UNIT TESTS RUN ON EACH NONTRIVIAL FUNCTION.....	29

1.0 Introduction

Folding letters into thirds for envelopes may seem like a simple task, but it requires precision, consistency, and time, especially when scaled up beyond a few sheets. It quickly grows from a minor annoyance into a tedious, error-prone bottleneck when hundreds of letters need to be prepared. In office, administrative, and mailing environments, manual folding increases variability, such as misaligned edges, uneven creases, and damage to the letter, all of which hurts professionalism and slows down workflow. By automating this task through a robot designed to fold letters into thirds, it eliminates these inconsistencies, reduces manual labour, and ensures a polished result every time. Addressing this problem is important because improving reliability and efficiency in letter preparation directly supports an organization's productivity, presentation, and overall operation standards.

1.1. Scope

The robot is a paper folding robot. The intention behind the functionality for the robot is for it to fold paper into thirds. More specifically, perfect thirds for letters that can then be loaded into envelopes. Originally, the idea was to build a robotic arm. This idea fell through due to technical limitations of VEX components, mainly from a lack of motors. The final design used a set of rollers to feed and fold the paper, as well as automatically adjustable hard-stops that allowed for different fold lengths. The motor encoders are used to determine the hard-stop position, the distance sensor detects when a piece of paper is fed into the folding rollers, the optical sensor is used to detect if the output bin is full, the touch LED is used to accept the hard stop positions and start the robot's folding, as well as confirming once a full output basket is cleared, and stopping the program when a jam is detected or if there is no more paper getting fed into the rollers, such as the input tray being empty.

1.2. Constraints

Certain project constraints existed limiting the final design. Resources were a major factor in building the robot as it could only use electronics from the provided VEXIQ robot. This was limited to 3-4 motors, and 1 of each available sensor. The robot was also required to receive 4 different types of inputs.

Further constraints were introduced to increase the focus of the final design. As folding letters was the main objective of the project, the final design is constrained only to support up to letter size paper. It was also only necessary for the robot to make 2 folds in the sheets of paper which is enough to fit them into envelopes. These folds were required to be customizable to support different envelope sizes. To increase efficiency the robot had to be capable of folding large stacks of paper by itself to reduce human

interaction. A maximum of 50 sheets of paper was set as a constraint to ensure this basic function of the robot was met while not having to design a massive system. In a similar vein, the robot needed to include an easily removable output tray. It needed to hold up to 25 sheets before becoming full. This was an additional constraint added after early design stages after observing how much volume the folded paper occupied.

1.3. Criteria

Similarly, criteria were introduced to evaluate the success of the final design. Several of the criteria existed to judge the quality of the folded paper. The team aimed to have the resulting creases in the paper be within 0.5 cm of the desired distance along the length of the paper. Another goal was to have this crease be within 5° of the horizontal to ensure a straight fold. This criterion was relaxed from the original target of 2° as it was difficult to design a feeder system that provided consistent results using VEXIQ parts without paper catching along the edges. The final fold would also ideally stay folded in resting position to demonstrate the sharpness of the fold. Paper integrity was another big concern. In early testing, paper would often be damaged or creased in unwanted areas due to various issues. The ideal product would be completely undamaged and would only have creases in the desired locations. Beyond each individual sheet of paper, the whole process was to take place quickly. The goal was to fold paper at a rate of 1 sheet per 10 seconds.

Paper jamming arose as a common issue in testing. As a result, the robot needed to be able to alert the user to any jams that would interrupt the robot's normal use. There was also the potential for the indexer to incorrectly feed two sheets of paper. The original idea was to have the robot not respond in this case allowing for both sheets to go through and be folded to reduce human interaction. This however was removed as a criterion to allow further precision on the jam detection function for an overall better end product. The robot also needed to stop and alert the user when the output tray became too full to continue operating.

The robot had specific start and end conditions that it needed to fulfill. It was required to start within a reasonable period, needing to run within 5 seconds of the user confirming the fold distance. This was part of general goal of speed and efficiency. The robot was also required to end by itself when the input tray was emptied to minimize how much human intervention is needed.

2.0 Mechanical Design and Implementation

Multiple potential solutions were evaluated according to the project constraints and criteria, with particular emphasis placed on speed and accuracy. Each concept was assessed using a decision matrix to compare overall performance.

Table 1: Overall design decision matrix

	Hinged Design	Robot Arm	Two-Armed	Rollers
Speed	4	3	6	8
Accuracy	5	3	4	8
Adjustability	4	10	7	8
Efficiency	6	2	4	9
Total	19	18	21	33

Hinged design: The hinged approach presented significant challenges. Producing two folds per sheet required precise repositioning of the paper on the folding plate between folds. Achieving the necessary accuracy with only four available motors greatly increased mechanical complexity, reducing the practicality of this option.

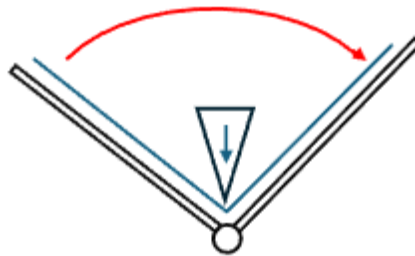


Figure 1: Hinge design

Robot arm: A single robotic claw or hand lacked the capability to reliably fold standard 8.5×11-inch paper. Even with dexterous motion, additional mechanisms would have been required to stabilize the sheet during folding. With only four motors available, creating a functional gripping system capable of consistent and controlled folding was not feasible.

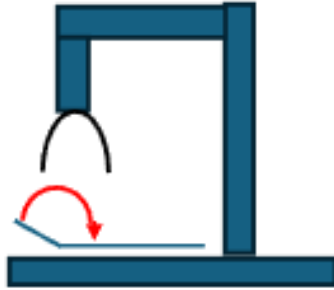


Figure 2: Robot arm

Two-armed design: In a dual-arm setup, each arm would have been limited to two motors, resulting in restricted degrees of freedom. As with the single-arm design, constructing an effective paper-handling gripper under these constraints would have been complex and likely unreliable.



Figure 3: Two-armed design

Roller design (chosen solution): The roller system was identified as the most effective solution. Continuous roller motion, combined with strategically positioned hard stops, enabled the paper to fold naturally at the desired locations. The design required only one motor to power all rollers, allowing the remaining motors to be dedicated to the paper indexer and the two adjustable hard stops. This configuration provided the speed, accuracy, adjustability, and efficiency needed to meet all performance requirements.



Figure 4: Roller design

2.1. Frame and Bracing

A major factor in the reliability of the robot was a supportive frame; without a frame capable of holding all integral components in place, the robot would fail to successfully operate on a regular basis. The robot's mechanical parts were functionally divided into two main components: The feeder tray, and the folders. An output basket was also created, however no mechanical components relied on its frame for use.

The feeder tray was especially crucial to ensure the frame was stable, as without a stable frame, the paper was not feeding into the folding rollers correctly, causing it to jam, produce unwanted fold angles, or crinkling near the edges. Due to the frequent iterations and adjustments to the feeder tray, the frame ended up becoming a mess of pieces (see Figure 5), with remnants of previous prototyped ideas getting left behind. This caused the frame to end up with critical errors; the VEX parts were being severely bent and put under stress, leading to the robot having additional forces on it that were not necessary, and were causing the paper to feed incorrectly.

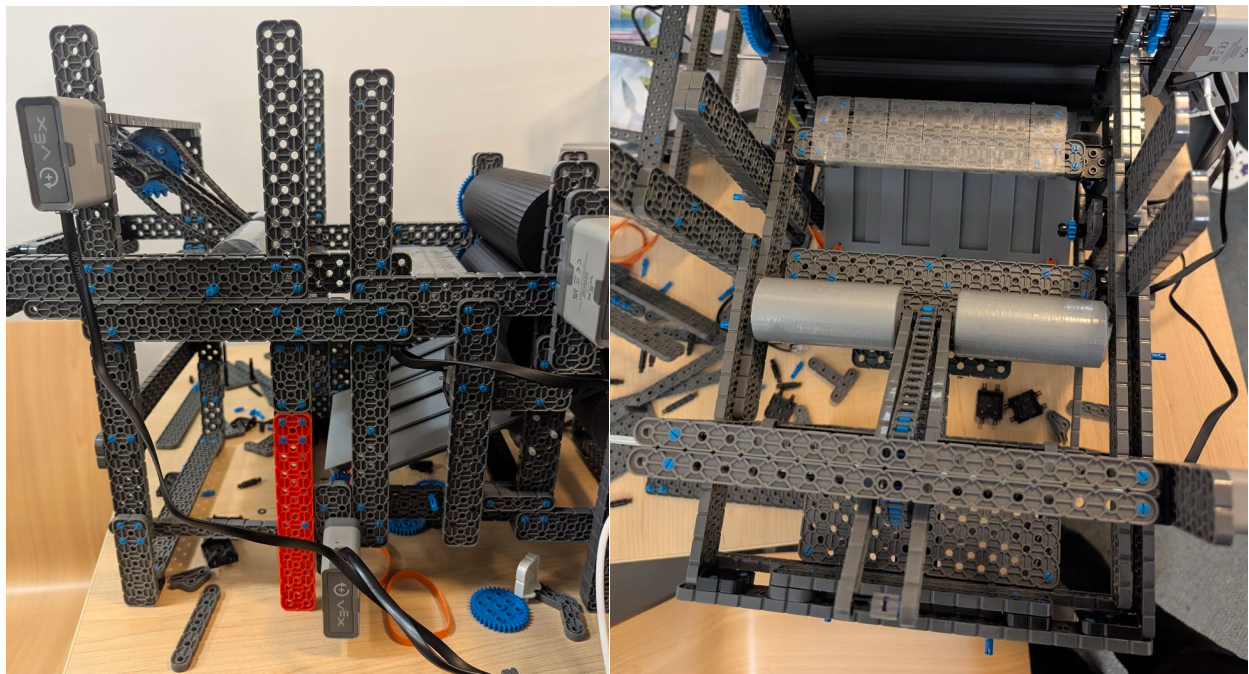


Figure 5: Old Frame Side View and Top View

To combat this issue, a full overhaul of the feeder tray's frame was executed (see Figure 6), making this section stable, symmetrical, and the exact width of a sheet of paper, as well as reducing the strain on the VEX parts.

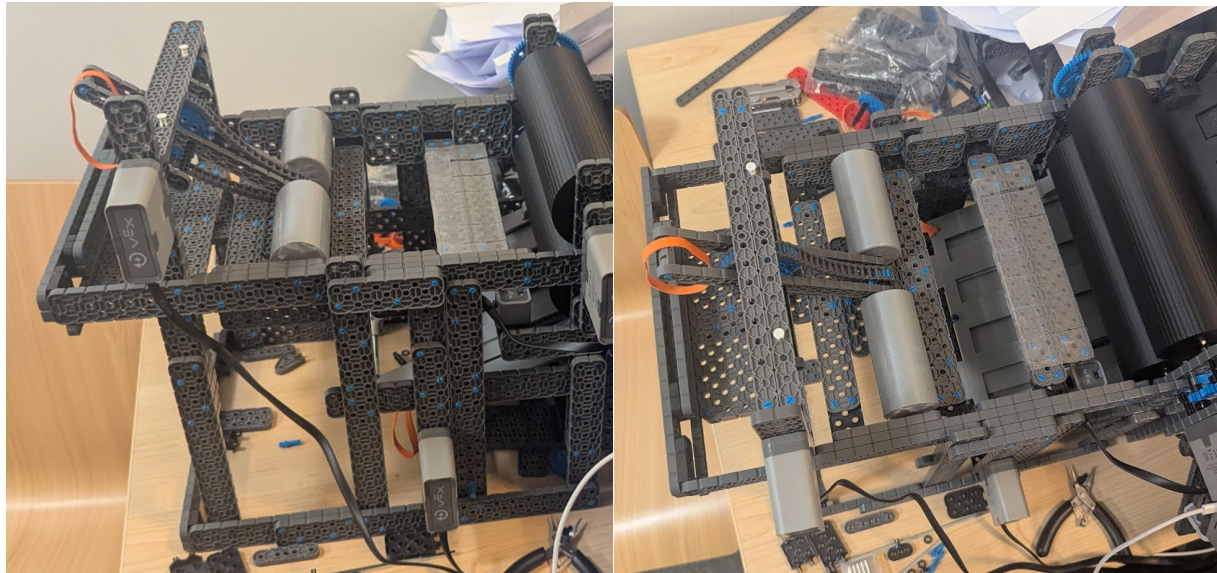


Figure 6: New Frame Side View and Top View

The folding section of the robot was made in CAD (see figures in Section 2.3 and Section 2.4) with the pieces and bracing meticulously selected for optimal structure and reliability in mind. Due to the clear planning that was done, the structure was strong and able to support the rollers. However, the shafts for the hard stops often failed to stay in place due to the frame bowing out. To correct this issue, bracing was added to give the shafts more contact points to the frame. (see Figure 7)

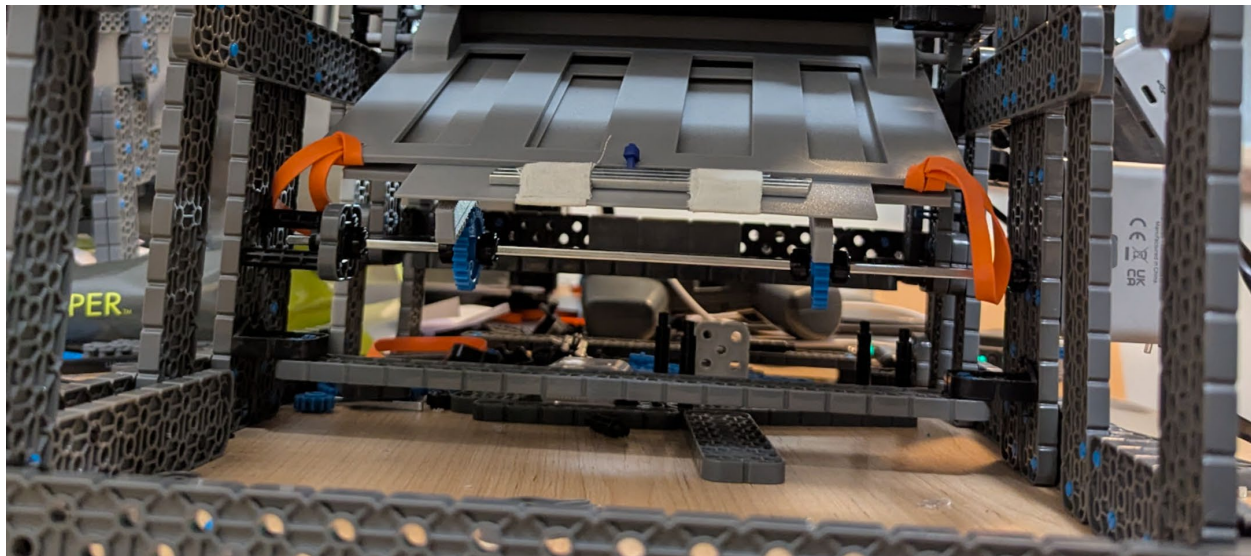


Figure 7: Shaft Bracing for Hard Stop Pinions

Another feasible solution to this problem would have been to use longer shafts, however due to limited resources, some of the shafts the team had access to were not long enough to function reliably without the use of additional bracing.

2.2. Indexer

The feeder tray and indexing system went through several iterations before it could consistently index a single sheet of paper.

The initial approach involved a roller lying beneath the stack of paper that would try and feed the bottom sheet of paper. This was wrapped in rubber bands to increase the friction between the bottom sheet of paper and the roller. This also involved a compressive force from the top to maintain pressure and help the bottom roller grip the paper. The downward force was a combination of the weight of the compressor and rubber bands pulling it down. Friction was also added to the top of the stack to try and prevent the roller from indexing the whole stack. This design choice was inspired by the separation pads used in certain printers as they similarly provide resistive friction to separate the bottom sheet from the stack [1]. This is seen below in Figure 8.



Figure 8: Initial bottom feeding indexer design using rubber bands for grip

There were many issues with this approach. The total downward pressure was not consistent as the stack of paper was consumed. This led to situations where there were too much pressure and friction between the sheets of paper which led to the roller either not indexing any paper or trying to index the whole stack. Maintaining the differences in friction between the sheets of paper is also a particularly difficult task, even for home printers [1].

This prompted the idea to try and increase the friction between the bottom sheet of paper and the roller while keeping the friction between the sheets of paper the same. Thus, improving the roller's grip became the next goal. A decision matrix was used to determine how to approach improving the roller's grip from using rubber bands.

Table 2: Decision matrix for rubber band replacement

	Non-Lubricated Polyurethane condoms	Polyurethane Molded Roller	Hot Glue Coated Cylinder
Feasibility	9	2	5
Grip	8	9	7
Durability	4	8	5
Total	26	19	17

Using the decision matrix from Table 2, using non-lubricated polyurethane condoms seemed to be the best replacement for rubber bands. The original idea was to use a custom mold polyurethane roller, but that proved to too expensive and unavailable during the building process of the robot. Hot glue was proposed as a possible coating to the rollers. Even though this was a feasible option, it was too difficult to implement into a reliable solution. Polyurethane condoms proved to be the most available option commonly sold as an alternative to latex condoms. Being the same material as the original idea of a polyurethane mold it provided reliable performance with any concerns over structural integrity being a very minor compromise.

However, the problem of inconsistent pressure was one that could not be resolved through more grip alone. It required a redesign of the feeding system to be top feeding instead of bottom feeding. This made it easier to keep the downward pressure consistent as the changing weight of the stack of paper is no longer a factor. The design relied on the rollers being suspended from a fixed point and driven using a chain and sprocket system. Rubber bands were used to add downward pressure but could be tuned more easily with the updated design. This combined with the previous improvement to grip resulted in consistency in indexing a single sheet of paper no matter the stack size. By consequence, the resistive friction that was previously on top of the stack was moved to the bottom to serve purpose. The final roller design can be see below in Figure 9.

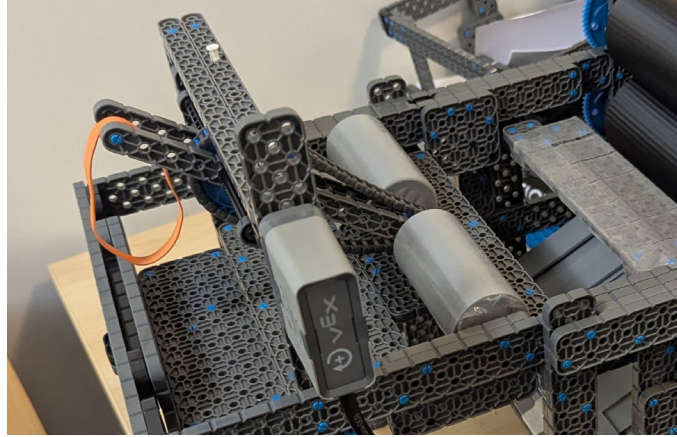


Figure 9: Final top feeding indexer design using polyurethane condom for grip

2.3. Rollers

The rollers were designed in CAD to ensure that their geometry, including their spacing and diameter, would allow for precise and consistent paper handling. All four rollers were powered by a single motor using a system of 60T gears, which provided continuous rotational movement. In operation, the paper was guided through the first two rollers until it reached the first hard stop. It then moved through the second and third rollers to create the first fold. After contacting the second hard stop, the sheet passed through the final pair of rollers to form the second fold.

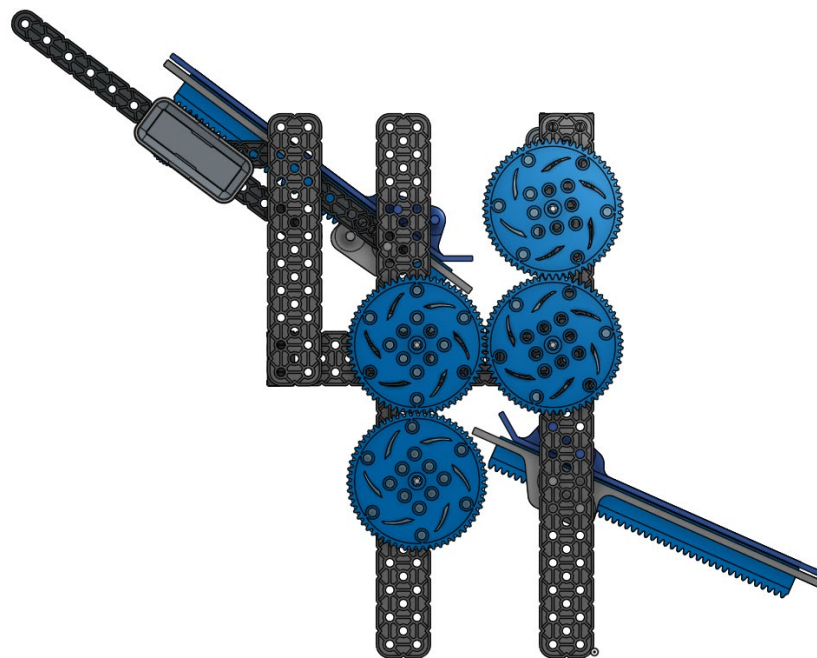


Figure 10: Roller gear layout CAD

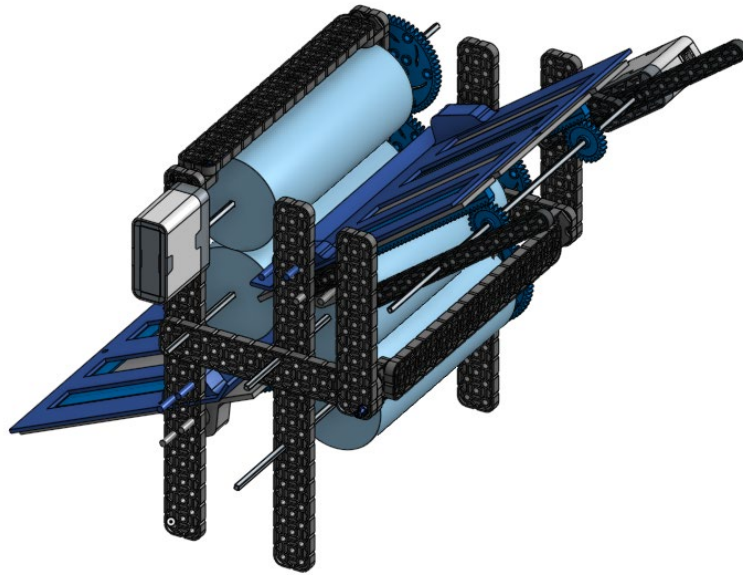


Figure 11: Roller CAD

Initial testing used the 200 mm VEX tires supplied in the kit. These tires did not provide enough surface contact with the paper, which resulted in folds that were not crisp or consistent. Testing then shifted to fully 3D-printed rollers, which offered complete contact along the paper's surface and produced much more accurate folds. During the transition from the first fold to the second, the 3D-printed rollers did not generate enough grip. To solve this problem, a hybrid setup was introduced. The third roller used a VEX tire hybrid to increase traction, while the remaining rollers stayed fully 3D-printed. This combination provided smooth transitions, reliable grip, and consistent fold quality.

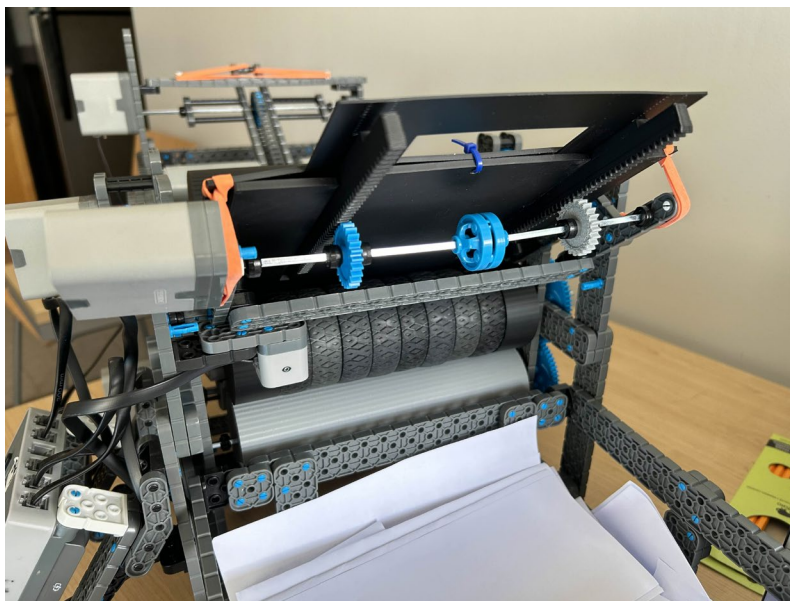


Figure 12: Hybrid roller

2.4. Hardstops

Precise control of each hard stop was necessary to ensure that the folds were placed within 0.5 cm of the desired location. Since only one motor was available for each hard stop and the folding assembly had limited internal space, the mechanism needed to be compact, accurate, and capable of linear motion.

Three viable mechanisms were evaluated in the decision matrix in Table 3: a crankshaft, a lead screw, and a rack and pinion system.

Table 3: Hardstop mechanism decision matrix

	Crankshaft	Lead Screw	Rack and Pinion
Accuracy	5	9	7
Range of Motion	1	7	8
Compactness	9	5	8
Complexity	6	3	10
Total	21	24	33

Crankshaft: The crankshaft design had a very limited and nonlinear range of motion. Because the displacement of a crank mechanism follows a sinusoidal curve rather than a linear one, achieving fine positional control would have required significant gear reduction. This would have slowed the mechanism while still providing only modest accuracy. These characteristics, outlined in standard crankshaft design explanations, made the crankshaft unsuitable for the level of precision required for the hard stop positioning [1].

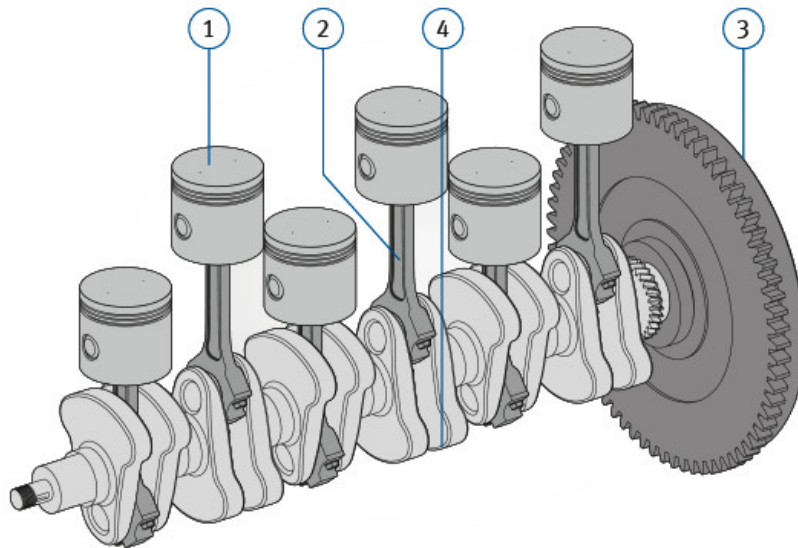


Figure 13: Crankshaft [1]

Lead screw: A lead screw offered good accuracy but did not fit within the available space. The screw length required for the desired range of motion would have interfered with the roller assembly and surrounding structural components. Lead screws also require adequate support, careful alignment, and higher torque at the motor, especially under load, as noted in industrial engineering references. Given the limited materials available and the torque constraints of the VEX IQ motor, this mechanism was not practical for the system [2].

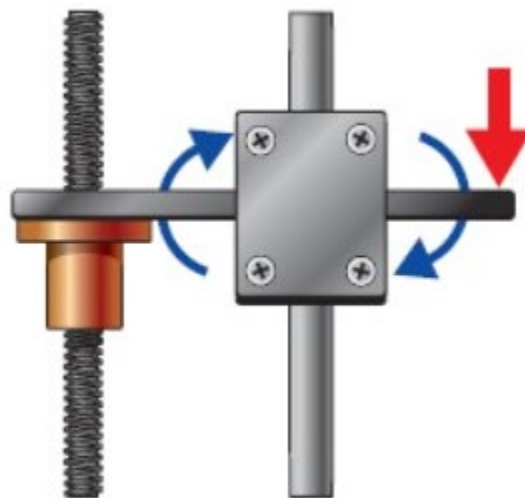


Figure 14: Guided lead screw axis [2]

Rack and pinion (chosen solution): The rack and pinion mechanism provided the best combination of accuracy, range of motion, and compactness. The motor, axle, and pinion gear could be placed away from the roller assembly, which prevented interference with other components. A 3D-printed rack could be

extended easily to achieve the needed linear travel. The system offered fast and reliable positioning while remaining simple to construct. For these reasons, the rack and pinion mechanism met the project's spatial constraints, material limitations, and precision requirements more effectively than the alternatives.

The rack and pinion system was used to move a hard stop linearly within the tray that guided the paper before it entered the rollers for folding. The assembly was designed in CAD to achieve precise tolerances and to ensure smooth paper movement. The rack was fully 3D printed and was driven by two 24T gears, which provided sufficient torque for reliable motion. During testing, the flexibility of the 3D printed parts caused the rack to lose consistent engagement with the pinion. To correct this issue, holes were added and rubber bands were used to apply downward tension, which kept the pinion and rack in steady contact. A small zip tie was also placed at the end of the tray to act as a physical stop, preventing the rack from sliding out during operation.

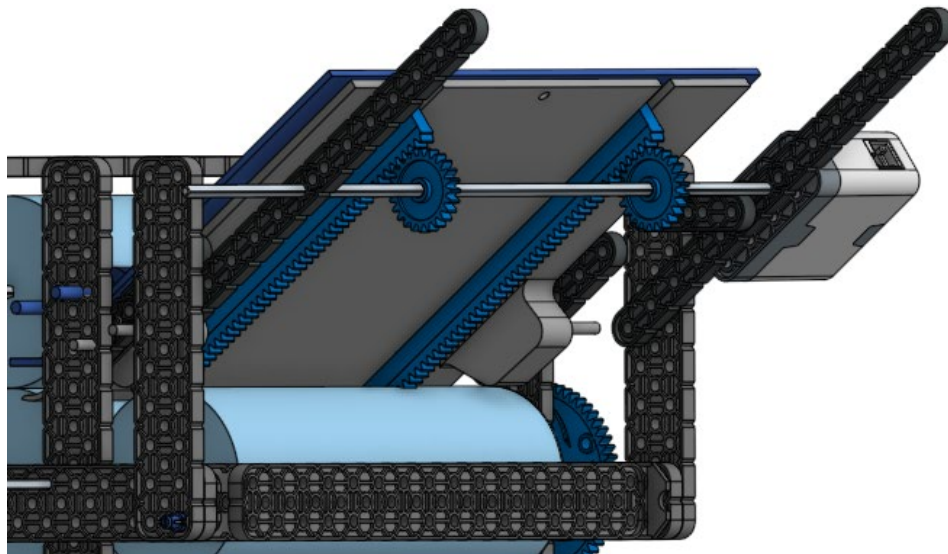


Figure 15: Rack and pinion tray CAD

A mirrored version of this mechanism was built for the second fold, although a new problem appeared. The paper approached the tray at a much steeper angle, which caused it to bounce off the entrance rather than slide in smoothly. One possible solution was to change the overall angle of the tray, but this would have required raising the entire system to create enough ground clearance. It also would have required reprinting several major components rather than modifying only the tray itself. Instead, the entrance of the tray was redesigned with a gentle curve to guide the paper downward. This approach required minimal reprinting and fit within the existing height constraints. The curved entrance significantly improved the paper's transition into the tray and fully resolved the issue.

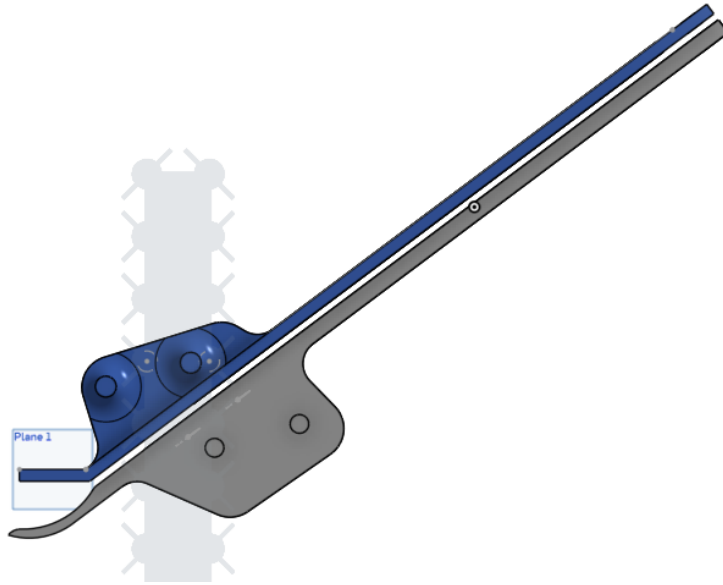


Figure 16: Curved tray CAD

2.5. Output Tray

After the paper is folded, the rollers would spit the paper out. This required a collection system that took the form of a detachable basket, allowing for an easy way to clear the robot of paper. The design needed to be easy to attach and detach, hold the tray in place while in use, and be sturdy enough to hold the paper, be picked up and cleared, and replaced with little to no difficulty.

A way to lock the basket in place temporarily was to use magnets placed into the holes of the VEX parts, contained by 3D printed caps using an interference fit. (see Figure 17)

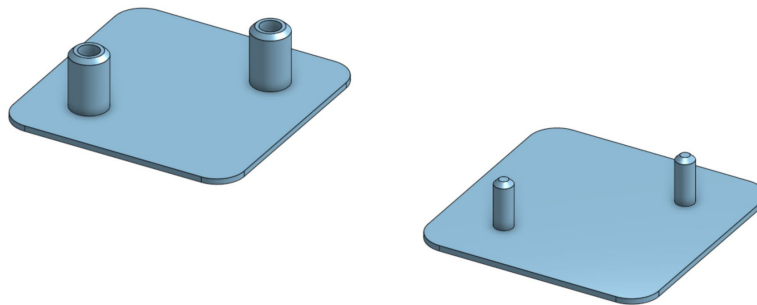


Figure 17: Magnet holders

These magnets held the basket in place securely, while simultaneously establishing simple removal and clearance. The original installation of the magnets had them placed on the sides of the basket, which narrowed the overall width of the slot the basket fell in to, which caused it to catch and not sit correctly

during the installation process. Additionally, this design had the basket sitting too far from the rollers, making the paper fail to fall properly into the basket. A redesign was completed, placing the magnets on the front of the basket, which made installation easier, put the basket closer to the rollers, and eliminated all major issues of the original design.

The basket itself required four legs, bracing to prevent paper from falling through, and enough height and depth to allow for the collection of a significant number of folded sheets. (see Figure 18) As there were few technical components to the basket frame, the final design remained mostly unchanged from the original design.

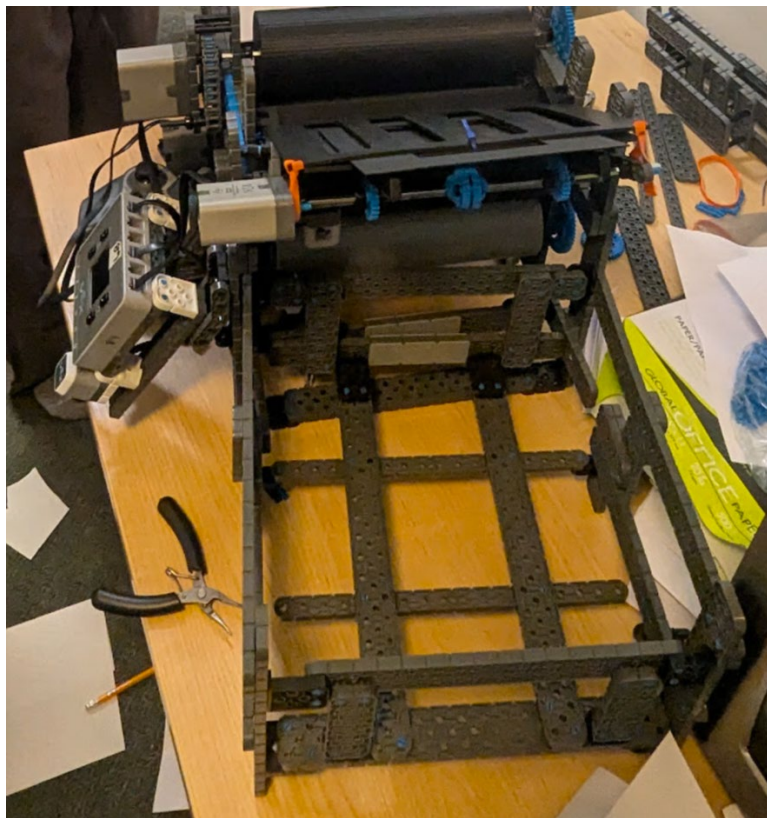


Figure 18: Completed Output Tray in Place

3.0 Software Design and Implementation

The software was written in C++ using the VEXCode IQ library, organized using the object-oriented programming paradigm. Classes were organized into files, each with its own header and source file. Generally, each physical subsystem was broken into its own class, including a motor class. Separate functions were also created to declutter the amount of code in `main()`.

The lack of multithreading also led to most functions being iterative rather than a blocking function. This means that all while loops that perform an action that last a significant amount of time stayed within `main()`. This made it easier to add disruption features, such as a pause, as discussed in Section 3.2.1.

3.1. Fold Location Input

Table 4 provides a summary of this function, noting the author of it.

Table 4: Function Summary of `inputFoldLocations`

Function	<code>inputFoldLocations</code>
Description	Waits for user input through the controller and touchLED, validates the input, and returns the fold locations.
Return Type	Void
Parameters	Fold 1 Location (double&): Reference to the variable to return the first fold location. Fold 2 Location (double&): Reference to the variable to return the second fold location.
Author	Jonathan Wang

This function takes the input from the user through the controller and returns the two fold locations to `main()`. Additionally, it also performs input validation, ensuring the system can perform folds at the inputted locations. It communicates the current fold locations, as well as any errors or warnings, through the LCD display on the brain. This function terminates on the press of the touch LED, the E up controller button, or the check brain button, where it returns the two fold locations using pass-by-reference.

Utilizing arrays of pointers to the buttons and to the fold location references, for loops are used to reduce the repetitiveness of each button. This is achieved by taking advantage of the indices through integer division, modulus, and its parity. This is seen in Figure 19. The $i / 2$ converts the button index to the fold location index. This is required because the buttons array is four buttons long, whereas there are only two fold locations. The $i \% 2$ checks the parity of the index. If it is even, then it should decrement whereas if it is odd, then it should increment the fold location. The $(i / 2 + 1) \% 2$ gets the other fold location index. The modulus 2 in this case does not check for parity but instead wraps it around so it

does not exceed the size of the array. `Util::sgn()` returns the sign of the value, which is either +1 for positive values, -1 for negative values, and zero for zero. By multiplying by the sign of $1 - i$, it shifts the other fold location towards the other direction of the movement of the current fold location. This line is to prevent the folds being too close together it being an impossible fold.

```
if (btns[i]->pressing() && brain.Timer.value() - pressSt[i] >= 0.5)
{
    if (i % 2)
        *foldLocs[i / 2] -= precision;
    else
        *foldLocs[i / 2] += precision;

    if (*foldLocs[1] - *foldLocs[0] < hardstop1.getMinFoldPos())
    {
        *foldLocs[(i / 2 + 1) % 2] = *foldLocs[i / 2] +
            hardstop1.getMinFoldPos() * Util::sgn(1 - i);
    }
}
```

Figure 19: Code snippet of using arrays to increment/decrement fold locations

Input validation consists of ensuring the folds are within the length of a Letter sheet of paper, as well as that the folds are within the minimum and maximum fold locations. The software follows these constraints and does not allow the user to input invalid fold locations. The software also ensures the first fold is before the second fold location. In other words, the second fold will adjust accordingly if the first fold moves too close to the second fold and vice versa. The last input validation that is performed is the overlap of folds. When the folds are too close together, the folds will overlap, causing three folds instead of two. Instead of disallowing this, the system instead warns the user, so they are aware of this occurring. More details about the mechanical system of the hardstop system is discussed in Section 3.6, including about the fold location limits and the fold overlap.

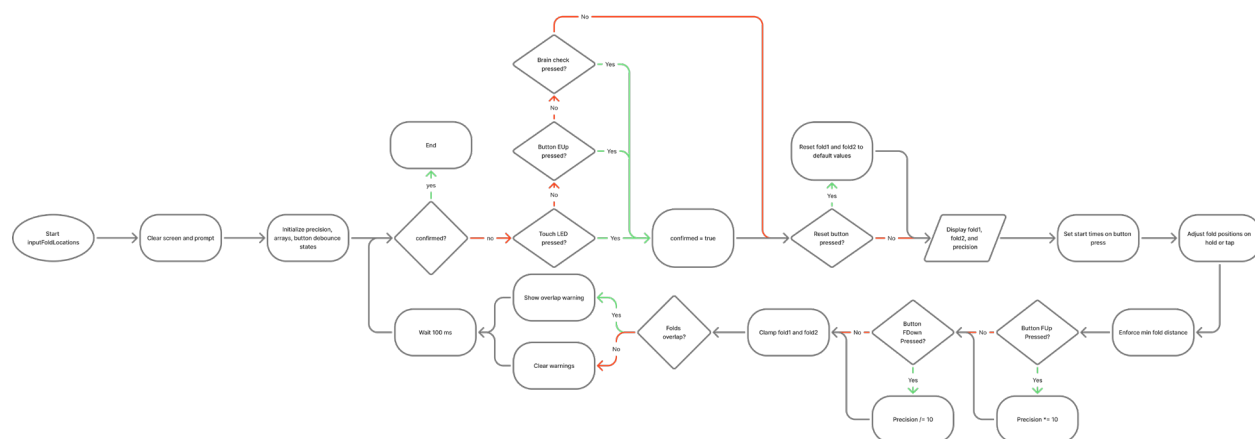


Figure 20: Flowchart of InputFoldLocations()

3.1.1. Button Presses

This function also handles all the button presses of the controller, touch LED sensor, and brain. One issue is that only the `Button.pressing()` method is allowed to be used. This means A debounce is required to perform actions on the rising edge or the falling edge of a button press. This is done by keeping track of a boolean variable that is true when the button is pressed and false when the button is not pressed. The code then detects when the boolean variable changes from false to true, essentially detecting the rising edge. The opposite can be done for detecting the falling edge by simply inverting the true and false states.

Additionally, as seen in Figure 20, the button presses can also handle taps and holds to increment and decrement the fold positions. This is done by keeping track of the time when the button is pressed. Then, if the button is released before 0.5 seconds has passed, then it is considered a tap, and the fold position is incremented or decremented a single time. If the button is still being held down after 0.5 seconds, then it is considered a hold, and the fold position is incremented or decremented every iteration of the loop until the button is released.

3.2. Main Process

The main process is where the system creates the classes and calls the methods discussed in the other sections. Generally, this method is a lot simpler since a lot of it has been abstracted behind the classes and methods. However, because of the lack of multithreading, the iterative approach requires the while loops to be inside `main()` as it allows for other iterative methods to be run in the same loop. This can be seen in Figure 21, where multiple methods are being called every iteration of the loop.

One such example is checking when the feeder tray is empty. This is achieved using a timeout when the feeder is indexing a sheet of paper. If after three seconds of feeding, the distance sensor has not detected a sheet of paper, then the system knows the feeder tray is empty and terminates both loops.

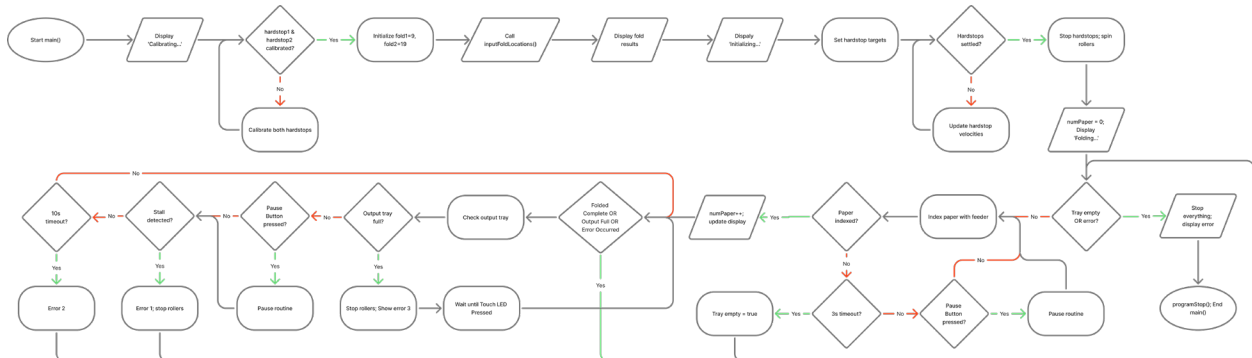


Figure 21: Flowchart of Main()

3.2.1. Pause

One of the exceptions to the iterative functions is the pause function. This function is blocking as it stops all motor movements and then waits until the pause button is pressed again. Instead of using the debounce technique discussed in Section 3.1.1, the pause function simply uses a while loop to wait until the button is pressed or released. This makes the function not iterative, but blocking, which is what is required of this function.

The function starts off with waiting for the pause button to release. This makes detecting the pause button press easier in `main()` since no debounce is required this way. The moment the button is pressed, this function blocks the program from running and so the button being held is not detected again since it does not check again until the `pause()` function completes and unblocks the process, at which point it has already made sure the pause button has been released.

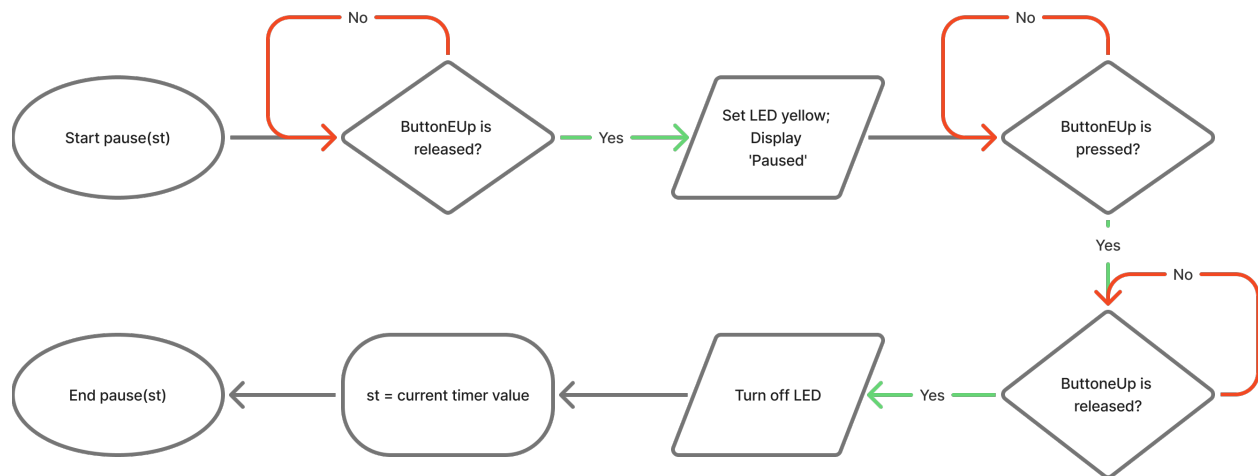


Figure 22: Flowchart of Pause()

It also resets the timeout by setting the start time to the time when the system is resumed. This is done using pass-by-reference, allowing it to reset different timeouts depending on the variable that is passed in by the parameter and the context it is called.

Table 6 provides a summary of this function, noting the author of it.

Table 5: Function summary of Pause()

Function	Pause
Description	Blocks the current program and waits for the unpaue button to be pressed.
Return Type	Void

Parameters	Start Time (Double&): Resets this at the end of pause to ensure timeouts do not take the paused time into consideration.
Author	Noah Sun

3.2.2. Display Error

Another function that blocks and is not iterative is the `displayError()` function. This function uses a switch-case on the error code variable that is kept track throughout `main()`. Similar to the `pause()` function from Section 3.2.1, this function blocks by listening for the touch LED to be pressed and released using while loops instead of debounces.

As shown in Figure 23, the only valid error codes are 0 to 3, so there is no default behaviour for the switch-case statement as the error code will never be outside this interval.

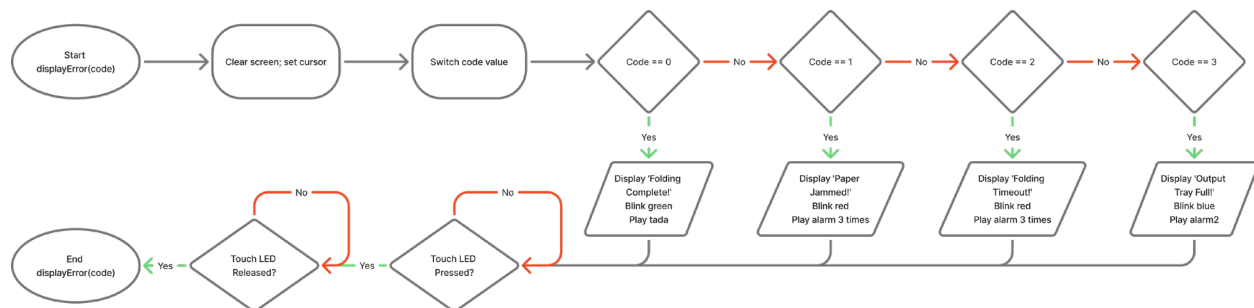


Figure 23: Flowchart of DisplayError()

Table 6 provides a summary of this function, noting the author of it.

Table 6: Function summary of DisplayError()

Function	displayError
Description	Takes the current error code and alerts the user accordingly
Return Type	Void
Parameters	Error Code (int): The current error code to display
Author	Noah Sun

3.3. PID Controller

A PID controller is a closed-loop control system that has a large variety of applications to help make systems move accurately and efficiently. Since it is a closed-loop control system, it is also known as a feedback loop. This means it updates in real time, reacting to sensor inputs, making it useful when unexpected scenarios occur. The controller outputs a signal, typically bounded between -1 and 1. This control value is then multiplied by the normal output value of the system, creating a modified value that reacts in real time to different situations.

3.3.1. Theory

PID stands for proportional, integral, and derivative, representing the three main components that make up the control output. All these portions use the error of the current system, which is the difference between the current value, known as the process variable (PV), and the target value, or setpoint (SP).

The proportional term simply makes the control signal proportional to the error. If the error gets smaller, the control signal gets smaller. If the error gets bigger, the control signal gets bigger. This creates a deceleration effect as the system moves closer to the desired state, allowing for more consistent behaviours.

However, an issue that can occur with this is called steady-state error. The steady-state error is when the error is so small that the proportional term is unable to overcome the minimum power required to move the system, but the PV has not yet reached the SP, causing the system to not be considered as settled. This is where the integral term comes in. It is the integral of the error over time. The idea of the integral term is that by the time the proportional term is extremely small, the integral would have accumulated enough where the integral term overpowers the proportional term, allowing the system to overcome steady-state error. While the integral term solves one issue, it also has its own problems. One such problem is known as integral windup. This is when the integral term accumulates a large error, causing a significant overshoot and a slow recovery. This occurs when the system output is at its maximum or minimum (the system is known as saturated), which can cause the integral term to accumulate error very quickly and becoming very large and causing an overshoot. When it overshoots, the error flips its sign and so the large integral term needs to be unwound, leading to an inefficient response. Integral windup can be overcome in many ways. One way is to tune the integral term very carefully, which is discussed further in Section 3.3.2. Other ways are to set a maximum and minimum integral term or to only start accumulating the integral term within a range of the SP. Another method that helps is to reset the integral term when the PV crosses over the SP. A combination of all the above-mentioned methods can be used as well to minimize integral windup.

Another issue that occurs is called PID oscillations. As the name suggests, it is where the PV oscillates around the SP until it finally settles to a slow enough velocity. This occurs due to the system overshooting the SP and then repeatedly overcorrecting. To combat this problem, the derivative term is introduced. It is the derivative of the error and is meant to smoothen out the deceleration created by the proportional term. It achieves this by applying a negative value on the control value since the error should be decreasing due to the proportional term. As the proportional term gets smaller, the effect of the derivative term becomes more noticeable, causing a dampening affect on the control variable and the error. Similar to the integral

term, the derivative term also introduces its own problems. Taking the derivative can amplify any noise that are present in the error from sensor readings. This is mitigated by applying a low-pass filter on the error, such as an exponential moving average filter.

Putting these three terms together, (3.1) shows the mathematical calculation of the control value as a function of time. The final control variable $u(t)$ is determined by the weighted sum of the proportional, integral, and derivative terms.

$$u(t) = K_p e(t) + K_i \int_0^t e(\tau) d\tau + K_d \frac{de(t)}{dt} \quad (3.1)$$

where K_p , K_i , and K_d are non-negative real numbers, known as the gain factors of each term. These are the coefficients of each term and are determined by the user, typically through trial and error. It also gives more control over how the PID controller reacts to changes, which is different between every system, depending on its environments and a variety of other factors. This is discussed further in Section 3.3.2.

3.3.2. Tuning

The three parameters K_p , K_i , and K_d , are known as the PID gains and are the coefficients of their respective terms. They act as a weighting in the final output, representing how much of an effect that specific term will have on the system's response to change. The values of the gains vary between every system and the process of finding the correct gains for each system is known as tuning. Some systems will also not require all three control terms. Certain systems may only need one or two terms and are named based on the terms that it uses. For example, a controller that only uses the proportional and integral term is known as a PI controller and is used when the sensor reading is too noisy.

The point of tuning is to optimize the controller, making it as responsive as possible while minimizing overshoot. However, these two contradict each other as an oversensitive controller will overshoot while a system that ensures no overshoot will be slower in how it responds to change. This means the goal of tuning is to find the equilibrium between these two extremes.

The process of tuning a PID controller is mainly through trial and error. In industry, there are autotuning algorithms that are used to generate preliminary gains, but that still requires human tuning afterwards. Such autotuning algorithms are outside the scope of this project and report. Focusing on the human tuning process, there are techniques that can aid in this process. The K_p is generally increased until the system oscillates slightly before settling. This ensures that the system settles efficiently. Then, K_d is increased to reduce the oscillations so it only overshoots once or twice before settling. Afterwards, if the system has a steady-state error, K_i can be increased to overcome the steady-state error. Adding the K_i will most likely

require decreasing the K_p and increasing the K_d , though. A step can be skipped, which turns off that term by setting a gain of zero. This reduces the controller to only use one or two control terms.

3.3.3. Implementation

The main implementation of the PID controller is an iterative `calculate()` method. It takes in the current error and will return the current control variable. There is an overloaded `calculate()` method that takes in two variables instead, the PV and the SP, but that simply calls the original `calculate()` method after calculating the error as $SP - PV$.

As can be seen in Figure 24, the integral windup mitigation techniques implemented were to only accumulate the integral when the PV is within a certain range of the SP. Additionally, resetting the integral on the PV crossing the SP helps minimize the affect of integral windup. The algorithm checks if the system has crossed the SP by comparing the sign of the current error with the sign of the previous error. If the sign is not the same, then it has crossed the SP and will reset the accumulated integral.

Another thing to note is that no derivative filter was applied. After some testing, the noise in the motor encoders seemed to not be a big issue. A possible explanation could be that the VEX IQ motors simply cannot respond quickly enough for the noise in the derivative term to affect the output.

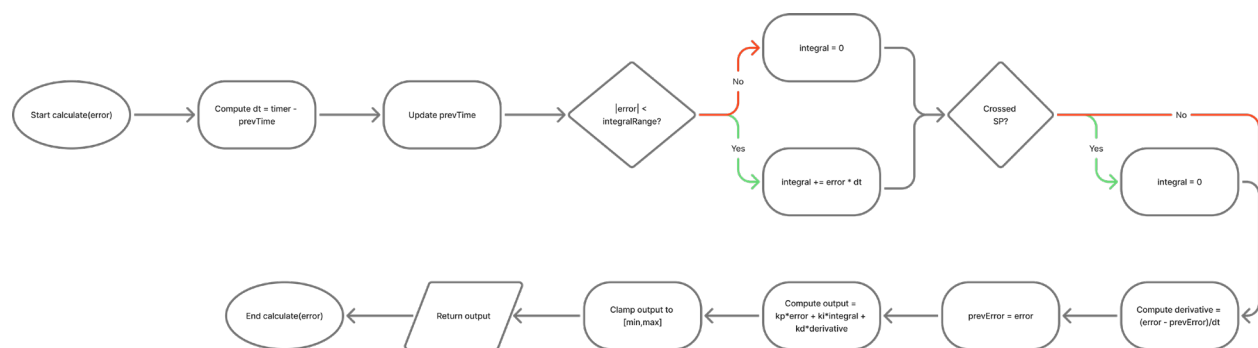


Figure 24: Flowchart of PID.Calculate()

Table 7 provides a summary of the function, noting the author of it.

Table 7: Function summary of PID.Calculate()

Function	PID.calculate
Description	Calculate the control value output of the PID controller given a process variable and setpoint.
Return Type	Double
Parameters	PV (double): The process variable, or current measurement, of the system SP (double): The setpoint, or target measurement, of the system
Author	Connor Zhou

3.4. Motor

The motor class is a custom class that publicly inherits the VEX motor class. This means the class provides all the functionality of the VEX motor class but additional features can be added. The two additional features that were added was to check if the motor is stalling and to track the counts per revolution. Stall detection will be discussed further in Section 3.4.1.

A tick is the smallest unit of rotation the encoder can measure. This measurement is preferred over degrees or rotations due to precision. By working with the raw encoder ticks, more precision can be achieved and skips over the conversions between ticks and degrees or rotations. According to the [4], the VEX IQ motor encoder has 960 ticks per revolution. This is simply stored as a constant static member variable of the class and is used when calculating distances.

3.4.1. Stall Detection

Stall detection is the main reason for creating a custom motor class. To determine if the motor is stalling, two criteria must be met: the current velocity is a given threshold away from the target velocity and that the current being drawn is greater than a given threshold as a percentage of the maximum current.

Calculating the velocity error requires to track the target velocity. This is achieved by creating a method in the motor class overloading the `setVelocity()` method. All it does is set a member target variable to the target velocity and then calls the VEX motor `setVelocity()` method. When checking for stalls, the method simply compares the absolute difference between the current velocity with the target velocity with a given threshold. If it is greater, then it may be considered as stalling. By default, this value is 40 RPM, but different applications with different desired sensitivity will have different thresholds.

Initially, simply monitoring the velocity was used to detect stalls, but the issue that occurred was the motor would be considered as stalling when accelerating up to the target velocity. However, the VEX Library Knowledge Base [4] states that the motor has an internal PID loop that controls “output speed and rotation”. What this means is that the PID loop will increase the current the motor is receiving to bring the velocity up to the target velocity. This means spikes in the current will most likely mean the motor has run into significant resistance and most likely has stalled. Generally, a spike where the current being drawn is greater than 50% of the maximum current, then it can be considered as stalled. However, different applications, depending on the desired sensitivity, will have different values.

Combining these two measurements gives a very consistent detection of stalls in the VEX IQ motors. It is required that both are above their respective thresholds for the motor to be considered as stalling. Table 8 provides a summary of this function, noting the author of it.

Table 8: Function summary of Motor.IsStalling()

Function	Motor.isStalling
Description	Detect when a motor is stalling or jammed.
Return Type	Boolean
Parameters	Velocity Threshold (double): The velocity difference from the current velocity and the target velocity for the motor to be considered as stalled. Current Threshold (double): The percentage of the max threshold for the motor to be considered as stalled.
Author	Jonathan Wang

3.5. Feeder

The feeder class inherits the motor class, which is detailed in Section 3.4. Additionally, it also has the VEX distance sensor and the timer. The constructor passes in the motor, distance sensor, and timer using references. This ensures there only exists one object of each physical sensor and motor. Most importantly, the timer object is the same throughout all classes as this ensures all classes are synchronized. The motor is initialized using the copy constructor of the VEX motor class.

3.5.1. Index

The main task of the feeder is to index one sheet of paper. This method is iterative rather than blocking. By initializing the start time of the indexing as a static variable, the length of time the distance sensor has detected the sheet of paper can be calculated and so it can be tuned to the proper timing.

Additionally, after it has indexed one sheet of paper, it will reverse the motor at a slower speed and torque to ensure the next sheet of paper starts at the same position every time. This is because sometimes, the next sheet of paper in the stack might stick slightly to the top sheet due to static friction and get dragged along. By lowering the torque and velocity, it will smoothly return the sheet to the top of the stack without damaging the paper. This can be seen in Figure 25.

This method will also return true once the paper has been indexed while returning false when it is not complete. This allows it to be the condition of the while loop, increasing readability.

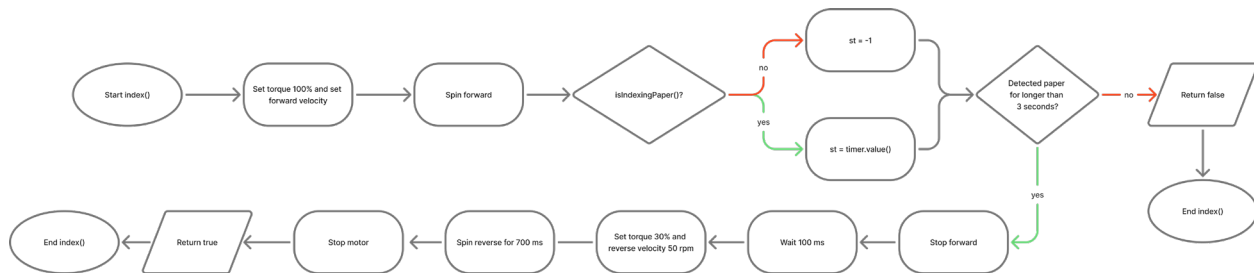


Figure 25: Flowchart of Feeder.Index()

Table 9 provides a summary of this function, noting the author of it.

Table 9: Function summary for Feeder.Index()

Function	Feeder.index
Description	Feeds exactly one sheet of paper into the rollers.
Return Type	Boolean
Parameters	Power (double): The velocity the feeder will spin at.
Author	Jakob Patel

3.6. Hardstop

The hardstop class inherits the motor class, while also having member variables for a PID controller object and the timer. More details about the motor class in Section 3.4. Details about the PID controller itself is in Section 3.3 while how it is used for the hardstop is discussed in Section 3.6.2.

Additionally, each hardstop has its own minimum and maximum fold positions, which are found by physically moving the hardstop to its minimum and maximum positions, folding a sheet of paper, and then measuring where the folds end up on the paper. Each hardstop also stores its maximum encoder value, which is when the hardstop is at the maximum fold position. The minimum encoder value is not necessary as it will always be zero, found using the `calibrate()` method, discussed in Section 3.6.1.

3.6.1. Calibration

Since knowing the positions of the hardstops is required to know where to fold, the system will reset the hardstops to a predictable position and set that as the new zero position. Calibrating the system is crucial to the robot being able to fold accurately.

The system finds the predictable position by moving the hardstops until they cannot move anymore. This is detected by the `Motor.isStalling()` method, discussed in Section 3.4.1. Once the motor has reached the end and has stalled, it resets the motor encoder to zero so this position is the new reference point. It also sets a calibrated flag so it does not keep on trying to calibrate after it has already been calibrated. As seen in Figure 26, if the hardstop has already been calibrated, `calibrate()` just immediately exits and does nothing.

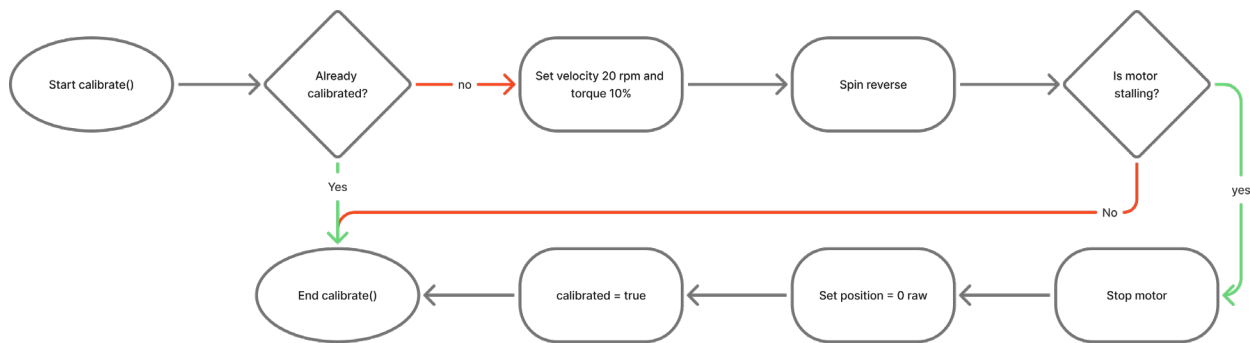


Figure 26: Flowchart of Hardstop.Calibrate()

Table 10 provides a summary of this function, noting the author of it.

Table 10: Function summary of Hardstop.Calibrate()

Function	Hardstop.calibrate
Description	Calibrates the hardstop, finding its zero position.
Return Type	Void
Parameters	N/A
Author	Noah Sun

3.6.2. Control

Controlling the actual hardstops to ensure it would move to the target positions used the `setTarget()` and `update()` methods.

The `setTarget()` is a method that takes a fold position and converts it into a motor encoder tick value. It also takes a maximum power as an RPM. The fold position is mapped to motor encoder ticks as a linear relationship. Using the minimum and maximum fold position and the maximum encoder position, the slope is calculated. The relationship is then horizontally translated to the right by the minimum fold position. Putting this together, this gives (3.2).

$$post_{tgt}(fold) = \frac{enc_{max} - enc_{min}}{fold_{max} - fold_{min}}(fold - fold_{min}) \quad (3.2)$$

The `update()` is an iterative method that moves the hardstop from its current position to the target position using a PID controller. How the PID controller works is explained in Section 3.3. It applies the PID before spinning the motor by multiplying the control output by the maximum power. The method also does nothing if the hardstop is already settled, which is when the current encoder reading is within a positional threshold of the target position and the velocity is below a velocity threshold.

3.7. Rollers

The rollers is a class that inherits the motor class discussed in Section 3.4. Additionally, it also stores a reference to an optical sensor and the timer. While the folding motion of the roller is simply spinning the motors, the rollers also detect for jams using the `Motor.isStalling()` method, explained fully in Section 3.4.1.

The optical sensor, using the built-in infrared sensor, can detect the sheet of paper exiting the rollers, indicating the fold has been completed. This uses the same logic as the button debounces from discussed in Section 3.1.1, however instead of button presses, it is whether the optical sensor has detected a sheet of paper. The previous state of whether a paper has been detected is stored using a static boolean variable in

the method. It is looking for the falling edge, where the paper is not detected in the current method call but the previous method call did detect the paper. When this occurs, it means the paper has fully left the rollers and the next sheet of paper can be indexed. Table 11 provides a summary of this function, noting the author of it.

Table 11: Function summary of Rollers.IsOutputTrayFull()

Function	Rollers.isOutputTrayFull
Description	Detect whether the output tray is full.
Return Type	Boolean
Parameters	N/A
Author	Connor Zhou

The optical sensor can also be used to detect when the output tray is full. The idea behind this is that the newly folded paper will get stuck at the exit of the rollers, being detected by the optical sensor, because the stack of folded paper in the output tray blocks the newly folded paper. This will mean the optical sensor detects the paper for an extend period time. This means a timeout can be used. If a sheet of paper is detected for greater than a time threshold, then this means the paper is stuck on something, most likely the full stack in the output tray. Table 12 provides a summary of this function, noting the author of it.

Table 12: Function summary of Rollers.IsClear()

Function	Rollers.isClear
Description	Detect whether the sheet of paper has been completely folded
Return Type	Boolean
Parameters	N/A
Author	Connor Zhou

3.8. Task List

Table 13 is the final tasks list of the system. There was only one change to this list, which was the addition of detecting the full output tray and pausing the system to allow for the user to empty the tray. The size of the output tray was not known initially, so the need to detect when the output tray is full was a requirement appended later.

Table 13: Final tasks list used during Demo Day

Category	Task
Start Up	Calibrate and tare the hardstops
Start Up	Input fold locations with input validation
Start Up	Initializes the hardstops to the correct positions
Main Process	Feed exactly one sheet of paper into the rollers

Main Process	Folds the sheet of paper twice
Main Process	Feeds the next sheet of paper once the previous sheet has been folded and ejected into the output tray
Main Process	Pauses when the output tray is full to allow the user to empty it and resumes once the touch LED is pressed
Shut Down	Detects the empty feeder tray
Shut Down	Alerts the user that all the paper has been folded and exits the program
Unexpected Case	Detects jams and alerts the user to fix the issue
Unexpected Case	Detects when rollers are slipping and alerts the user to fix the issue

3.9. Testing Procedure

The software was run through unit tests initially to ensure each method worked individually. This made it easier once everything was put together, during integration testing, where all the functions were tested working together.

Table 14 lists all the unit tests that were run on each function and their use and expected test results. Tests were monitored to ensure they met the expected results through constant data logging to the terminal and through careful inspection using measuring tapes to make sure they are within tolerance. By logging data to the terminal, it can be pasted into an Excel Spreadsheet to generate a graph, where the data can be visualized to assist in debugging.

Table 14: Unit tests run on each nontrivial function

Function	Unit Tests	Test Results
inputFoldLocations	Increment/decrement by tapping the buttons	When a button is pressed for less than 0.5 seconds, the respective fold location only increments/decrements once per press.
	Increment/decrement by holding the buttons	When a button is held for more than 0.5 seconds, the respective fold location will continuously increment/decrement until the button is released.
	Increment/decrement the precision to affect the increment/decrement size of each fold location	Changing the precision will increase or decrease it by a power of 10. Incrementing/decrementing fold locations will increase/decrease by the precision amount.

	Proper input validation is applied on the fold locations	Ensures the two fold locations are the minimum distance away and both fold locations are valid locations on the sheet of paper.
pause	Properly blocks the program and resumes on button press	Ensures pause stops other loops from going through iterations until the program is resumed again.
displayError	Displays the correct error message for each error code	Prints the correct error message on the brain, flashes the touch LED the correct colour, and plays the correct sound effect depending on the error code.
Pid.calculate	Test by adding one gain at a time.	By starting with the proportional term, it is easy to spot any logic errors straight away. By adding the integral term and derivative term individually, bugs in the integral term and the derivative term can be spotted separately.
	Call with variable Δt in between calls	Since the Δt is calculated by the function, this needs to be accurate or else the integral and derivative terms will not be properly calculated.
Motor.isStalling	Use a thick sheet of paper to simulate a jam in the rollers	This is the main use case for this function so testing if it can detect a jam will be important.
	Spin a motor normally with occasionally a little resistance with no false triggers of stalling	The function cannot have false positives where it thinks a small resistance that does not actually cause the motor to stall as stalling.
Feeder.index	Feeds exactly one sheet of paper	Uses the distance sensor and proper timing and tuning to feed exactly one sheet of paper into the system
	Successfully detects an empty feeder tray	When the feeder tray is empty, it detects it is empty using a timeout. This must also

		not have any false positives as it will result in a premature termination of the program.
Hardstop.calibrate	Successfully finds the zero position and stops the motor, then resets the encoder	Using <code>Motor.isStalling()</code> , the hardstop can consistently find the zero position that it will set as its new reference point.
Rollers.isOutputTrayFull	Successfully detects when the output tray is full	It can detect when the output tray is full and requires the user to empty it. This also cannot have false positives as it will result in the system pausing randomly
Rollers.isClear	Successfully detects when a sheet of paper has been folded	It can consistently detect when a sheet of paper has left the rollers to signal for the next sheet of paper to be indexed. This cannot have false positives as it will result in a sheet of paper to be indexed when one is already in the system, causing jams and inaccurate folds.

In terms of integration testing, it was conducted in a similar manner, where the tests were run with data being logged to the terminal. If something went wrong or was not within tolerance, then something required more tuning or the data visualization would reveal a bug in the logic. Integration testing was mostly the debugging of the logic in `main()`.

3.10. Problems Encountered

One of the biggest problems was the timing of the feeder. Due to the different diameters of the rollers, the paper would be moving at different speeds in the two different sections. The naïve solution implemented was to tune the feeder to stop exactly when the paper has entered the rollers. This required a lot of tuning and is still not completely consistent. Better solutions and approaches are discussed in Section 7.2.

Another problem that was already mentioned in Section 3.4.1 is the false positives that would occur when only monitoring velocity for detecting stalls. This was overcome by adding current to the requirements to be considered as stalling. This created an elegant solution that could be visualized by logging the data and using an Excel Spreadsheet, allowing for easier debugging.

4.0 Verification

The final robot design met all the updated criteria and constraints. It made the two required folds for fitting letter size paper into envelopes by having the two sets of hard stops and rollers. Both hard stops could be positioned at any distance up to 10cm making the robot capable of making most combinations of two folds. It also supported the folding of up to 50 sheets of paper through the redesign of the frame and the switch to a top feeding indexing system. The frame redesign helped performance because it was redesigned specifically to help feed paper straightly by being sized for letter sized paper. The robot also met all the materials and project constraints using only the given VEX electronics. It still however received the 4 different input types required of the robot receiving information from the motor encoders, distance sensor, optical sensor, touch LED and controller.

The robot was successful in meeting the goals set to evaluate the performance of the robot. The resulting folds passed all check of quality. They were within 0.5cm of the desired distance, within 5° of the horizontal, did not damage the paper, and the creases remained sharp in resting state. The robot successfully stopped and alerted users during jam tests when the sheets of paper were simulated to be stuck or slipping. It also successfully alerted the user when there was no longer any space in the output tray. The robot was more efficient than the mark of one sheet per 10 seconds, folding a sheet every 5 seconds. Similarly, the start up and calibration was faster than the mark of 5 seconds requiring only 2 seconds. Shut down procedure was also successful as the robot consistently detected when the input tray was empty using the optical sensor on the feeder.

Even though all the updated constraints and criteria were met, there remain many possible improvements for the robot design. Many of the criteria for fold quality could be made stricter for an improved final product in terms of fold quality. This would primarily require improvements to the folding rollers and the indexing system. Improvement to the efficiency and consistency could be made by targeting the tuning of the software and relying less on timers in the decision making of the software.

5.0 Project Plan

The project was organized using a Gantt chart that outlined the planned timeline, major milestones, and task durations. Responsibilities were divided so that each team member contributed to both mechanical and software components. The initial mechanical concept, including the roller mechanism and adjustable hard stops, was developed by Connor. Jonathan designed the paper indexer and the brain mount. Jakob developed the output tray and assisted with reinforcing the frame as mechanical demands increased. Noah led the software architecture, and programming tasks were shared among all members, with each person implementing specific functions such as the PID controller, jam detection, sheet indexing, hard stop control, and user input handling. This distribution ensured an equitable and balanced workload.

As development progressed, the project plan required several revisions. More mechanical issues arose than anticipated, particularly with paper alignment, roller grip, and hard stop motion, which led to multiple redesigns of the tray, frame, and rack-and-pinion assemblies. The removable output tray was also introduced later in the process after initial testing showed it would significantly improve reliability. These changes pushed mechanical completion beyond the planned dates, causing software development and mechanical troubleshooting to overlap more than the original Gantt chart predicted.

The differences between the planned and actual timeline revealed several lessons about the development process. Mechanical redesigns showed that the robot needed to remain functional whenever possible so that software development could continue in parallel. Maintaining a clean workspace proved essential, as folding tests generated clutter that hindered assembly and debugging. The team also recognized the value of developing mechanical subsystems earlier and in parallel rather than waiting for one component to approach completion before beginning another. These reflections demonstrated a deeper understanding of how planning, scheduling, and workflow coordination could be improved in future projects.

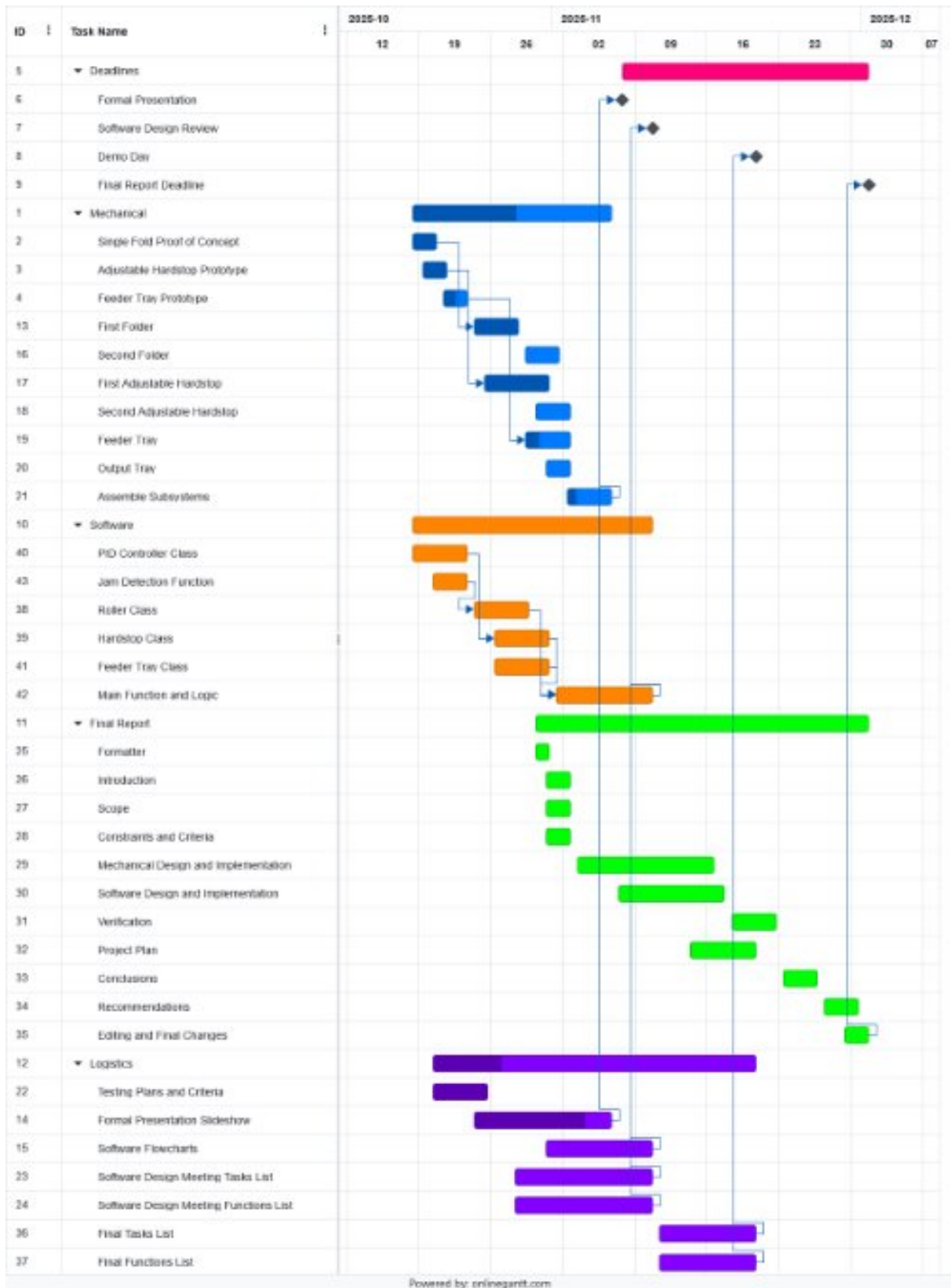


Figure 27: Initial Gantt chart

6.0 Conclusions

Overall, the paper folding robot was a success. The problem of having to fold a sheet of paper into equal thirds en masse while maintaining efficiency and consistency is something not feasibly attainable by a human, but the robot was able to perform this operation repeatedly and quickly. The criteria of having folds be within 0.5 cm of their desired location, as well as within 5° of the horizontals were comfortably met in each paper. Furthermore, jam detection was successful in registering malfunctions when they arose, and the papers were being folded under the desired rate of a single sheet per 10 seconds as outlined in the criteria. The robot fully operated under the constraints outlined by the team and the project itself, ensuring that there were no additional electronics required to make the robot function, as well as meeting the required amounts of sensors and working within the constraint of a maximum of four motors.

The mechanical design of the robot was divided into three main sections: the feeder, the folder, and the output tray. The feeder tray and indexer used a top roller design, which indexed the top sheet of paper forward towards the folding section while keeping all other loaded sheets in place. The top rollers were attached to an arm that consisted of a chain and sprockets and was driven by a motor. The folding section consisted of two main components, the rollers, and the hard stops. Three of the four rollers were entirely 3D printed, whereas one roller used 3D printed end caps with VEX wheels in the middle. This grippy roller was in position for both folds, assisting the paper in being fed through the system. The four rollers were driven by a single motor in a 1:1 ratio between them. The two hard stops were built entirely of three 3D printed parts; a top, bottom, and sliding piece were designed to create modular folding locations driven by a rack and pinion. The sliding piece was printed with two racks, and had teeth designed to mesh with 24T gears. Each of these sets of pinions were driven by their own motor, enabling the option for custom fold locations. After exiting the folding section, the papers were collected by an output tray, which was held in place using magnets that were imbedded into the VEX parts and held in place by 3D printed caps. The output tray could hold 25 sheets of paper, as outlined in the criteria, and was easy to remove, empty, and reinstall.

The software design of the robot included crucial elements that allowed the robot to function efficiently and precisely. One such element of the software that was key in meeting the criteria was the inclusion of a PID controller. The PID controller was used for the hard stops, making sure that the fold locations were within the required 0.5 cm as outlined in the criteria. Additionally, the distance and optical sensors were used to detect paper at different points throughout the folding process. The distance sensor was coded to detect when a piece of paper was fed into the folding rollers and would recognize when new paper was no longer being fed in, signalling that there was no more paper left in the feeder tray. The optical sensor was

used at the output tray and would recognize when a sheet of paper has left the system. By having the optical sensor checking for papers, this created a detection point for the basket being full; if the basket was full, there would be paper in constant contact with the optical sensor, having an unbroken detection of the paper for an extended amount of time. The motor encoders were also used in multiple situations. During calibration, the hard stops would move to the shortest position, and the motor telemetry would reach a threshold that would indicate it has reached this point, and then the encoders would move to the default position of creating folds at perfect thirds. For the folding rollers, the motor sensors were used in detecting jams; if paper became stuck in the rollers, the motors would have to work harder, which was detected by the software and would automatically pause the program. The touch LED and controller worked in tandem with these sensors and was used as the human interaction point for anything needing confirmation, including starting the folding process from the beginning or after clearing the output tray, and exiting the program once the feeder tray was empty or if a jam was detected.

7.0 Recommendations

7.1. Mechanical Design

Verification testing showed that the roller system produced reliable folds, but occasional slipping slowed the folding process and reduced consistency. Increasing roller grip would significantly improve performance. A future iteration could use polyurethane-molded wheels, which would provide both a precise diameter and full surface contact while offering much higher traction than the current 3D-printed rollers. This material would preserve the geometric accuracy needed for consistent folding while eliminating the slippage observed during testing.

The paper indexer experienced similar limitations. Although the improvised non-lubricated condoms provided adequate grip, they lacked durability and required washing to remove residual lubrication. Custom polyurethane molded rollers would be a more robust alternative for the indexer as well, providing longer lifespan, consistent grip, and reduced risk of tearing. Chamfering the indexer rollers also proved beneficial in preventing the condoms from ripping, and this design feature should be retained in future versions.

Initial mechanical testing also highlighted alignment issues caused by the flexibility of the VEX IQ structural components. The frame tended to deform under load, which affected the straightness of the paper path and ultimately required a full rebuild. A more rigid chassis, either through a fully CAD-designed structure, increased use of 3D-printed components, or the addition of aluminum L-channels or extrusions, would greatly improve alignment and overall reliability. Ensuring that the mechanical structure remains straight and stable is essential for accurate and repeatable folding.

7.2. Software Design

The software right now is overly reliant on timeouts. This is not a consistent solution because of variability between each iteration and test. It would be more consistent if more sensors that are also more precise and accurate were available. The main part this problem is present in is when feeding the sheet of paper, where the distance sensor detects the paper a considerable distance away from the rollers, leading to the requirement of a timeout to find the perfect timing to stop the indexer.

Another part that contributed to this issue is the different diameters of the feeder and the rollers, leading to the papers moving at different speeds in each section. This emphasizes the need for perfect timing further. A solution to this could be to calculate the angular velocity of the feeder and rollers and ensure they spin at the same speed.

Additionally, if there were more time, a more accurate mapping formula from fold location to encoder ticks could have been derived from simply more trials and testing. An attempt to improve the accuracy of the `calibrate()` method would also help this issue. While it is not guaranteed to fix the issue, possibly adding a delay between stopping the motor and resetting the motor encoder could help mitigate the inconsistency.

In industry, a class for the controller and touch LED should be created to handle all the debounces of the buttons. This will cut away a lot of the repetitiveness when using controller button and touch LED presses. It will also make the code in `main()` and `inputFoldLocations()` much more readable.

References

- [1] Brother Industries, Ltd., *HL-L2310D / HL-L2350DW / HL-L2357DW / HL-L2370DN / HL-L2375DW: Online User's Guide*, 2018. [Online]. Available: [cv_hll2310d_uke_oug_c.pdf](#)
- [2] X-Engineer, "Crankshaft," X-Engineer.org. Available: <https://x-engineer.org/crankshaft/>.
- [3] HaydonKerkPittman, "Lead Screws 101," Haydon Kerk Pittman Learning Zone White Papers. Available: <https://www.haydonkerkpittman.com/learningzone/whitepapers/lead-screws-101>.
- [4] "Understanding IQ Smart Motor Performance," VEX Library. [Online]. Available: <https://kb.vex.com/hc/en-us/articles/18113746616724-Understanding-IQ-Smart-Motor-Performance>.

Appendix A – C++ Code

main.cpp

```
#include "vex.h"
#include "motor.h"
#include "feeder.h"
#include "hardstop.h"
#include "rollers.h"
#include "util.h"

vex::brain brain;
vex::controller controller;

Motor feederMotor(vex::PORT7, true);
Motor rollerMotor(vex::PORT1, false);
Motor hardstop1Motor(vex::PORT6, false);
Motor hardstop2Motor(vex::PORT8, true);
vex::distance distanceSensor(vex::PORT2);
vex::optical opticalSensor(vex::PORT5);
vex::touchled touchLed(vex::PORT4);

void inputFoldLocations(Hardstop& hardstop1, Hardstop& hardstop2, double& fold1,
double& fold2)
{
    brain.Screen.clearScreen();

    brain.Screen.setCursor(5, 1);
    brain.Screen.setFont(vex::fontType::mono15);
    brain.Screen.print("Press Touch LED");

    double precision = 1.0;

    double* foldLocs[2] = {&fold1, &fold2};
    const vex::controller::button* btns[4] = {
        &controller.ButtonLUp,
        &controller.ButtonLDown,
        &controller.ButtonRUp,
        &controller.ButtonRDown
    };
    double pressSt[4] = {};
    bool precisionDebounce[2] = {};

    bool ledPressed = false, confirmPressed = false, controllerPressed = false,
    confirmed = false;
    bool resetPressed = false;
    while (!confirmed)
    {
        if (touchLed.pressing() && !ledPressed)
            ledPressed = true;
        if (!touchLed.pressing() && ledPressed)
            confirmed = true;

        if (controller.ButtonEUp.pressing() && !controllerPressed)
```

```

        controllerPressed = true;
    if (!controller.ButtonEUp.pressing() && controllerPressed)
        confirmed = true;

    if (brain.buttonCheck.pressing() && !confirmPressed)
        confirmPressed = true;
    if (!brain.buttonCheck.pressing() && confirmPressed)
        confirmed = true;

    if (controller.ButtonEDown.pressing() && !resetPressed)
    {
        resetPressed = true;
    }
    else if (!controller.ButtonEDown.pressing() && resetPressed)
    {
        fold1 = 9;
        fold2 = 19;
        resetPressed = false;
    }

    for (int i = 0; i < 2; i++)
    {
        brain.Screen.setCursor(i + 1, 1);
        brain.Screen.clearLine();
        brain.Screen.print("Fold %d: %.1f cm", i + 1, *foldLocs[i]);
    }

    brain.Screen.setCursor(3, 1);
    brain.Screen.clearLine();
    brain.Screen.print("Precision: %.1f", precision);

    for (int i = 0; i < 4; i++)
    {
        if (btns[i]->pressing() && !pressSt[i])
            pressSt[i] = brain.Timer.value();
    }

    for (int i = 0; i < 4; i++)
    {
        if (btns[i]->pressing() && brain.Timer.value() - pressSt[i] >= 0.5)
        {
            if (i % 2)
                *foldLocs[i / 2] -= precision;
            else
                *foldLocs[i / 2] += precision;

            if (*foldLocs[1] - *foldLocs[0] < hardstop1.getMinFoldPos())
            {
                // i / 2 gives the current fold index (integer division)
                // (i / 2 + 1) % 2 gives the other fold index
                // Util::sgn(1 - i) gives +1 if i is 0 or 2 (increasing first
                // fold), -1 if i is 1 or 3 (decreasing second fold)
                *foldLocs[(i / 2 + 1) % 2] = *foldLocs[i / 2] +
                    hardstop1.getMinFoldPos() * Util::sgn(1 - i);
            }
        }
    }

```

```

    }
    else if (!btns[i]->pressing())
    {
        if (brain.Timer.value() - pressSt[i] < 0.5)
        {
            if (i % 2)
                *foldLocs[i / 2] -= precision;
            else
                *foldLocs[i / 2] += precision;

            if (*foldLocs[1] - *foldLocs[0] < hardstop1.getMinFoldPos())
            {
                // i / 2 gives the current fold index (integer division)
                // (i / 2 + 1) % 2 gives the other fold index
                // Util::sgn(1 - i) gives +1 if i is 0 or 2 (increasing first
                // fold), -1 if i is 1 or 3 (decreasing second fold)
                *foldLocs[(i / 2 + 1) % 2] = *foldLocs[i / 2] +
                    hardstop1.getMinFoldPos() * Util::sgn(1 - i);
            }
        }
        pressSt[i] = 0;
    }
}

if (controller.ButtonFUp.pressing() && !precisionDebounce[0])
{
    if (precision < 10)
        precision *= 10;
    precisionDebounce[0] = true;
}
else if (!controller.ButtonFUp.pressing() && precisionDebounce[0])
{
    precisionDebounce[0] = false;
}
if (controller.ButtonFDown.pressing() && !precisionDebounce[1])
{
    if (precision > 0.1)
        precision /= 10;
    precisionDebounce[1] = true;
}
else if (!controller.ButtonFDown.pressing() && precisionDebounce[1])
{
    precisionDebounce[1] = false;
}

fold1 = Util::clamp(fold1, hardstop1.getMinFoldPos(),
                    hardstop1.getMaxFoldPos());
fold2 = Util::clamp(fold2, hardstop2.getMinFoldPos() +
                    hardstop1.getMinFoldPos(), hardstop2.getMaxFoldPos()
                    + hardstop1.getMaxFoldPos());

if (fold2 - fold1 < fold1)
{
    brain.Screen.setCursor(6, 1);
    brain.Screen.clearLine();
}

```

```

        brain.Screen.print("Folds overlap:");
        brain.Screen.setCursor(7, 1);
        brain.Screen.clearLine();
        brain.Screen.print("Will cause 3 folds");
    }
    else
    {
        brain.Screen.setCursor(6, 1);
        brain.Screen.clearLine();
        brain.Screen.setCursor(7, 1);
        brain.Screen.clearLine();
    }

    wait(100, vex::msec);
}

void displayError(int errorCode)
{
    brain.Screen.clearScreen();
    brain.Screen.setCursor(3, 1);
    switch (errorCode)
    {
        case 0:
            brain.Screen.print("Folding Complete!");

            touchLed.setBlink(vex::color::green, 0.5, 0.5);
            brain.playSound(vex::tada);
            break;
        case 1:
            brain.Screen.print("Error: Paper Jammed!");

            touchLed.setBlink(vex::color::red, 0.5, 0.5);
            for (int i = 0; i < 3; i++)
            {
                brain.playSound(vex::alarm);
                wait(500, vex::msec);
            }
            break;
        case 2:
            brain.Screen.print("Error: Folding Timeout!");

            touchLed.setBlink(vex::color::red, 0.5, 0.5);
            for (int i = 0; i < 3; i++)
            {
                brain.playSound(vex::alarm);
                wait(500, vex::msec);
            }
            break;
        case 3:
            brain.Screen.print("Output Tray Full!");

            touchLed.setBlink(vex::color::blue, 0.5, 0.5);
            brain.playSound(vex::alarm2);
            break;
    }
}

```

```

    }

    while (!touchLed.pressing()) {}
    while (touchLed.pressing()) {}
}

void pause(double& st)
{
    while (controller.ButtonEUp.pressing()) {}

    touchLed.setColor(vex::color::yellow);
    brain.Screen.setCursor(4, 1);
    brain.Screen.clearLine();
    brain.Screen.print("Paused");

    while (!controller.ButtonEUp.pressing()) {}
    while (controller.ButtonEUp.pressing()) {}

    touchLed.off();
    st = brain.Timer.value();
}

int main()
{
    printf("\033[2J\n");

    Feeder feeder(feederMotor, distanceSensor, brain.Timer);
    Rollers rollers(rollerMotor, opticalSensor, brain.Timer);
    Hardstop hardstop1(hardstop1Motor, 0.02, 0.015, 0.0013, 7.2, 17.1, 1352,
                      brain.Timer);
    Hardstop hardstop2(hardstop2Motor, 0.019, 0.015, 0.002, 8.5, 17.4, 1262,
                      brain.Timer);

    int errorCode = 0;

    brain.Screen.setCursor(1, 1);
    brain.Screen.print("Calibrating...");
    while (!hardstop1.isCalibrated() || !hardstop2.isCalibrated())
    {
        hardstop1.calibrate();
        hardstop2.calibrate();
        wait(50, vex::msec);
    }

    double fold1Location = 9;
    double fold2Location = 19;
    inputFoldLocations(hardstop1, hardstop2, fold1Location, fold2Location);

    brain.Screen.clearLine(3);
    brain.Screen.clearLine(5);

    brain.Screen.setCursor(4, 1);
    brain.Screen.clearLine();
    brain.Screen.print("Initializing...");

```

```

hardstop1.setTarget(fold1Location);
hardstop2.setTarget(fold2Location - fold1Location);

while (!hardstop1.isSettled() || !hardstop2.isSettled())
{
    hardstop1.update();
    hardstop2.update();
    wait(20, vex::msec);
}

hardstop1.stop();
hardstop2.stop();

rollers.spin();

int numPaper = 0;
brain.Screen.setCursor(4, 1);
brain.Screen.clearLine();
brain.Screen.print("Folding... %d", numPaper);

double st = 0;
bool isTrayEmpty = false;
bool hasError = false;
while (!isTrayEmpty && !hasError)
{
    st = brain.Timer.value();
    while (!feeder.index(40) || !isTrayEmpty)
    {
        if (brain.Timer.value() - st >= 3 && !feeder.isIndexingPaper())
            isTrayEmpty = true;

        if (controller.ButtonEUp.pressing())
        {
            feeder.stop();
            rollers.stop();

            pause(st);

            rollers.spin();
            brain.Screen.setCursor(4, 1);
            brain.Screen.clearLine();
            brain.Screen.print("Folding... %d", numPaper);
        }

        wait(20, vex::msec);
    }

    if (!isTrayEmpty)
    {
        st = brain.Timer.value();

        numPaper++;
        brain.Screen.setCursor(4, 1);
        brain.Screen.clearLine();
        brain.Screen.print("Folding... %d", numPaper);
    }
}

```

```

bool outputFull = false;
while (!rollers.isClear() && !outputFull && !hasError)
{
    if (rollers.isOutputTrayFull())
    {
        outputFull = true;
        rollers.stop();

        displayError(errorCode = 3);

        errorCode = 0;
        st = brain.Timer.value();
        rollers.spin();
    }

    if (controller.ButtonEUp.pressing())
    {
        rollers.stop();
        feeder.stop();

        pause(st);

        rollers.spin();
        brain.Screen.setCursor(4, 1);
        brain.Screen.clearLine();
        brain.Screen.print("Folding... %d", numPaper);
    }

    if (rollers.isStalling(60, 70))
    {
        rollers.stop();
        hasError = true;
        errorCode = 1;
    }

    if (brain.Timer.value() - st >= 10)
    {
        hasError = true;
        errorCode = 2;
    }

    wait(20, vex::msec);
}

rollers.stop();
feeder.stop();
hardstop1.stop();
hardstop2.stop();

displayError(errorCode);

brain.programStop();

```

```
}
```

rollers.cpp

```
#include "rollers.h"
```

```
using namespace vex;
```

```
Rollers::Rollers(Motor& motor, vex::optical& opticalSensor, vex::timer& timer)  
    : Motor(motor), opticalSensor(opticalSensor), timer(timer)
```

```
{
```

```
    setBrake(brakeType::coast);
```

```
}
```

```
bool Rollers::isPaperDetected() const
```

```
{
```

```
    return opticalSensor.isNearObject();
```

```
}
```

```
bool Rollers::isClear() const
```

```
{
```

```
    static bool wasPaperDetected = false;
```

```
    if (!isPaperDetected() && wasPaperDetected)
```

```
    {
```

```
        wasPaperDetected = false;
```

```
        return true;
```

```
    }
```

```
    wasPaperDetected = isPaperDetected();
```

```
    return false;
```

```
}
```

```
bool Rollers::isOutputTrayFull() const
```

```
{
```

```
    static double st = -1;
```

```
    if (isPaperDetected())
```

```
    {
```

```
        if (st == -1)
```

```
            st = timer.value();
```

```
        else if (timer.value() - st >= TIME_THRESHOLD_SEC)
```

```
            return true;
```

```
    }
```

```
    else
```

```
    {
```

```
        st = -1;
```

```
    }
```

```
    return false;
```

```
}
```

```
void Rollers::spin()
```

```
{
```

```
    setMaxTorque(100, percentUnits::pct);
```

```
    setVelocity(80, velocityUnits::rpm);
```

```
    Motor::spin(directionType::fwd);
```



```
}
```

rollers.h

```
#pragma once
```

```
#include "vex.h"
```

```
#include "motor.h"
```

```
class Rollers : public Motor
```

```
{
```

```
private:
```

```
    vex::optical& opticalSensor;
```

```
    vex::timer& timer;
```

```
    const double TIME_THRESHOLD_SEC = 3;
```

```
public:
```

```
    Rollers(Motor& motor, vex::optical& opticalSensor, vex::timer& timer);
```

```
    bool isPaperDetected() const;
```

```
    bool isClear() const;
```

```
    bool isOutputTrayFull() const;
```

```
    void spin();
```

```
};
```

hardstop.cpp

```
#include "hardstop.h"
```

```
using namespace vex;
```

```
Hardstop::Hardstop(Motor& motor, double kp, double ki, double kd, double minFoldPos,  
double maxFoldPos, double maxEnc, vex::timer& timer)
```

```
    : Motor(motor), pid(kp, ki, kd, timer), minFoldPos(minFoldPos),  
    maxFoldPos(maxFoldPos), maxEnc(maxEnc), timer(timer)
```

```
{
```

```
    setBrake(brakeType::coast);
```

```
}
```

```
void Hardstop::setTarget(double foldPos, double maxPower)
```

```
{
```

```
    this->tgtPos = (maxEnc / (maxFoldPos - minFoldPos)) * (foldPos - minFoldPos +  
1.1);
```

```
    this->maxPower = maxPower;
```

```
}
```

```
void Hardstop::update()
```

```
{
```

```
    if (isSettled())
```

```
    {
```

```
        stop();
```

```
        return;
```

```
    }
```

```

    double ctrl = pid.calculate(tgtPos, position(rotationUnits::raw));
    setVelocity(maxPower * ctrl, velocityUnits::rpm);
    setMaxTorque(100, percentUnits::pct);
    spin(directionType::fwd);
}

bool Hardstop::isSettled(double posThreshold, double velThreshold)
{
    return fabs(position(rotationUnits::raw) - tgtPos) < posThreshold &&
           fabs(velocity(velocityUnits::rpm)) < velThreshold;
}

void Hardstop::calibrate()
{
    if (calibrated)
        return;

    setVelocity(20, velocityUnits::rpm);
    setMaxTorque(10, percentUnits::pct);
    spin(directionType::rev);

    if (!isStalling(10, 20))
        return;

    stop();
    setPosition(0, rotationUnits::raw);
    calibrated = true;
}

```

hardstop.h

```

#pragma once
#include "vex.h"
#include "motor.h"
#include "pid_controller.h"

class Hardstop : public Motor
{
private:
    PidController pid;

    double minFoldPos;
    double maxFoldPos;

    double maxEnc;

    double tgtPos = 0;
    double maxPower = 50;

    bool calibrated = false;

    vex::timer& timer;

public:

```

```

Hardstop(Motor& motor, double kp, double ki, double kd, double minFoldPos, double
        maxFoldPos, double maxEnc, vex::timer& timer);

void setTarget(double foldPos, double maxPower = 60);

void update();

bool isSettled(double posThreshold = 3, double velThreshold = 20);

void calibrate();
inline bool isCalibrated() const
    { return calibrated; };

inline double getMinFoldPos() const
    { return minFoldPos; };
inline double getMaxFoldPos() const
    { return maxFoldPos; };
};

```

pid_controller.cpp

```

#include "pid_controller.h"
#include "util.h"

PidController::PidController(double kp, double ki, double kd, vex::timer& timer,
    double integralRange, bool resetIntegralOnCross, double
    outputMax, double outputMin)
    : kp(kp), ki(ki), kd(kd), timer(timer), integralRange(integralRange),
    resetIntegralOnCross(resetIntegralOnCross), outputMax(outputMax),
    outputMin(outputMin) {}

double PidController::calculate(double error)
{
    static double prevTime = 0;
    static double integral = 0;
    static double prevError = 0;

    double dt = timer.value() - prevTime;
    prevTime = timer.value();

    if (fabs(error) < integralRange)
        integral += error * dt;
    else
        integral = 0;
    if (resetIntegralOnCross && Util::sgn(error) != Util::sgn(prevError))
        integral = 0;

    double derivative = (error - prevError) / dt;
    prevError = error;

    return Util::clamp(kp * error + ki * integral + kd * derivative, outputMin,
        outputMax);
}

double PidController::calculate(double target, double current)
{

```

```

    return calculate(target - current);
}

```

pid_controller.h

```

#pragma once
#include "vex.h"

class PidController
{
private:
    double kp, ki, kd;

    vex::timer& timer;

    double integralRange = 15;
    bool resetIntegralOnCross = true;

    double outputMax = 1, outputMin = -1;

public:
    PidController(double kp, double ki, double kd, vex::timer& timer, double
        integralRange = 15, bool resetIntegralOnCross = true, double
        outputMax = 1, double outputMin = -1);

    double calculate(double error);
    double calculate(double target, double current);
};

```

motor.cpp

```

#include "motor.h"

using namespace vex;

Motor::Motor(int32_t port, bool reverse)
    : motor(port, reverse) {}

void Motor::setVelocity(double velocity, velocityUnits units)
{
    tgtVel = velocity;
    motor::setVelocity(velocity, units);
}

bool Motor::isStalling(double velThreshold, double currentThreshold)
{
    return fabs(tgtVel - fabs(motor::velocity(vex::velocityUnits::rpm))) >
        velThreshold && motor::current(percentUnits::pct) > currentThreshold;
}

```

motor.h

```

#pragma once
#include "vex.h"

class Motor : public vex::motor

```

```

{
private:
    double tgtVel = 0;

public:
    Motor(int32_t port, bool reverse = false);

    void setVelocity(double velocity, vex::velocityUnits units);
    bool isStalling(double velThreshold = 40, double currentThreshold = 50);

    const static int TPR = 960;
};

```

feeder.cpp

```

#include "feeder.h"

using namespace vex;

Feeder::Feeder(Motor& motor, distance& distanceSensor, vex::timer& timer)
    : Motor(motor), distanceSensor(distanceSensor), timer(timer)
{
    setBrake(brakeType::coast);
}

bool Feeder::isIndexingPaper() const
{
    return distanceSensor.objectDistance(distanceUnits::mm) <
PAPER_DETECTION_DISTANCE_MM;
}

bool Feeder::index(double power)
{
    setMaxTorque(100, percentUnits::pct);
    setVelocity(power, velocityUnits::rpm);
    spin(directionType::fwd);

    static double st = -1;

    if (isIndexingPaper())
        st = timer.value();
    else
        st = -1;

    if (timer.value() - st >= 3 && st != -1)
    {
        stop();
        st = -1;
        wait(100, msec);

        setMaxTorque(30, percentUnits::pct);
        setVelocity(50, velocityUnits::rpm);
        spin(directionType::rev);
        wait(700, msec);
        stop();
    }
}

```

```

        return true;
    }

    return false;
}

```

feeder.h

```

#pragma once

#include "vex.h"
#include "motor.h"

class Feeder : public Motor
{
private:
    vex::distance& distanceSensor;
    vex::timer& timer;

    const static double PAPER_DETECTION_DISTANCE_MM = 80;

public:
    Feeder(Motor& motor, vex::distance& distanceSensor, vex::timer& timer);

    bool isIndexingPaper() const;

    bool index(double power);
};

```

util.h

```

class Util
{
public:
    static inline double clamp(double value, double min, double max)
    {
        if (value < min) return min;
        if (value > max) return max;
        return value;
    }

    static inline int sgn(double value)
    {
        return (value > 0) - (value < 0);
    }
};

```